CODEBASE STUDY GUIDE

ContentUP

Natural Video Upscaler for Kaggle

Every concept, pattern and technique in the project — what it is, where it lives in your code, why it is there, how to explain it in an interview, and when to reuse it.

Covers: `video_upscaler.ipynb`, `dev/engine.py`, `dev/build_notebook.py`, `dev/test_local.py`, `README.md`, `.gitignore`

Code snippets are copied automatically from the repository with their real line numbers.

Contents

How to use this guide 4

Part 1 — The big picture 5

- Project overview 5
- Tech stack 5
- Architecture 6
- Journey of a request: "upscale my video" 7
- What is NOT in this project (and how to answer if asked) 8

Part 2A — Interview favourites (★★★) 10

1. Streaming pipeline with subprocesses and pipes 11
2. Producer–consumer with threads and bounded queues 14
3. Checkpointing and resume 17
4. Atomic writes (write-to-temp, then rename) 20
5. Graceful degradation and automatic fallbacks 22
6. Retry with exponential backoff 25
7. Input validation 27
8. Custom exceptions and a friendly error strategy 29
9. Decorators (@friendly) 31
10. Strategy pattern and duck typing (two interchangeable engines) 33
11. PyTorch models and loading pretrained weights 35
12. Automated end-to-end testing with fake data and test doubles 38

Part 2B — Important concepts (★★) 41

13. Cleanup on failure: try / except / finally and context managers 42
14. Idempotency: safe to run again 44
15. Deadlines and graceful shutdown (the 12-hour budget) 46
16. Hashing settings into a cache key 48
17. Factory function + memoization (build the engine once) 50
18. Lazy loading and lazy installation 52
19. Facade functions and separation of concerns 54
20. Configuration with defaults (and notebook globals) 56
21. Sliding window with overlap (context frames) 58
22. Batching 60
23. Tiling with overlap ("halo" padding) 62
24. Mixed precision (fp16) with a safety net 64
25. GPU device handling and memory management 66
26. Closures and callbacks (the emit function) 68
27. Recursion as divide-and-conquer 70
28. Weight interpolation (adjustable denoise strength) 72
29. Residual connections (and PixelShuffle) 74
30. How the steady model sees motion: optical flow, warping and two-way propagation 76
31. Reading video metadata with ffprobe (defensive parsing) 79
32. H.264 encoding settings 81
33. Segments, concatenation and audio muxing 83
34. Colour spaces: RGB vs YUV, BT.601 vs BT.709 85
35. Resampling: make it 4× bigger, then shrink carefully 87
36. Generated notebook: a single source of truth 89
37. The Kaggle platform model (and environment detection) 91
38. Installing packages at runtime, version pinning and patching third-party code 93
39. Resource checks: disk space and quotas 95

Part 2C — Basics (★) 97
40. Callable objects (__call__) 97
41. Exact fractions for frame rates 97
42. Peek / push-back buffer (deque) 97
43. Logging to files and showing the tail 97
44. cudnn.benchmark 98
45. Skipping wasted work 98
46. Tensor layout and normalisation 98
47. Frame-exact seeking with ffmpeg's select filter 99
48. Constant vs variable frame rate (CFR / VFR) 99
49. HDR tone mapping 99
50. Rotation metadata 99
51. Film grain synthesis 100
52. Alpha blending and before/after comparison 100
53. Progress bars with time remaining (tqdm) 100
54. Human-friendly formatting helpers 100
55. Git basics: .gitignore, branches, commit messages 101

Part 3 — Honest review 102
Security review 102
Things I could improve ("If I had more time, I would…") 103

Part 4 — Interview kit 105
Explain your project in 60 seconds 105
Glossary 105

Cheat sheet — last-minute revision 109

How to use this guide

This guide walks through **your own code**. Every code box is copied automatically from your repository, and its caption shows the real file and line numbers, so you can open the file and find the exact spot.

How it is organised

- **Part 1 — The big picture:** what the project does, the tools it uses, how the parts connect, and one full "journey" through the code.
- **Part 2 — Concepts, most important first.** There are three groups:
  - ★★★ *Interview favourites*: things interviewers love to ask about.
  - ★★ *Important*: solid engineering you should be able to explain.
  - ★ *Basics*: shorter notes, good to know.
- **Part 3 — Honest review:** security, weak spots, and what you would improve.
- **Part 4 — Interview kit:** a 60-second pitch, a glossary, and a one-page cheat sheet (the very last page).

Each full concept follows the same 8 steps: *What it is → Where it is → Why we used it → How it works → What would go wrong without it → Alternatives → Interview prep → Future use.*

> **A word on honesty in interviews.** This project was written with heavy AI help, and that is fine to say. Interviewers respect "I built it with an AI assistant. Then I studied every part, tested it, and here is why each decision was made" far more than pretending. This guide gives you the "why". Your job is to make it yours: read the code next to this guide, change a setting and see what happens, and break something on purpose to see the error message.

Part 1 — The big picture

Project overview

ContentUP is a **free video upscaler** that runs inside a **Kaggle notebook** on a borrowed T4 graphics card (GPU). You upload a video, run the cells in order, and get back a bigger, sharper H.264 MP4 with the **original sound and frame rate**. Its main goal is a **natural look**: it controls denoising, can blend some of the original back in, can add film grain, and leaves face "beautification" off by default. It offers two AI models: **"fast"** (Real-ESRGAN, one frame at a time) and **"steady"** (RealBasicVSR, which looks at neighbouring frames so there is less flicker). It also **saves progress** while it works, so a long video can survive Kaggle's 12-hour session limit, and a **preview mode** lets you test settings on a short clip first.

Tech stack

Technology	What it does here	Why it was likely chosen
Python 3	The language all the code is written in.	It is the standard language for AI work, and Kaggle notebooks run Python.
Jupyter notebook (.ipynb)	The "app" you run: boxes of code (cells) run one by one.	Kaggle's free GPUs are only available through notebooks. No server or website is needed.
Kaggle	A free cloud computer with a T4 GPU, input/output folders and background runs.	It is free (about 30 GPU hours a week), which was your requirement.
PyTorch (`torch`)	Runs the neural networks on the GPU (or the CPU for tests).	It is the most common AI library, and the Real-ESRGAN and RealBasicVSR weights are PyTorch files.
NumPy (`numpy`)	Holds each video frame as a grid of numbers (height × width × 3 colours).	It is the standard "array" library, and PyTorch converts to and from it cheaply.
ffmpeg / ffprobe	Reads any video into raw pictures, writes H.264 MP4s, copies the audio, reads metadata.	It is the industry-standard free video tool, already installed on Kaggle.
Real-ESRGAN `realesr-general-x4v3`	The "fast" upscaling model (≈1.2 M parameters).	It is small and quick, and it comes with a "less denoise" twin model, so denoising can be adjusted smoothly.
RealBasicVSR	The "steady" video model (it uses optical flow and neighbouring frames).	It reduces flicker, and it was trained on messy real-world video.
GFPGAN (optional)	Rebuilds faces.	It is the best-known face restorer, but it is off by default because it can look fake.
tqdm	Progress bars that show the time left.	It is a one-line way to get a progress bar and ETA.

Technology	What it does here	Why it was likely chosen
OpenCV (`cv2` , optional)	Writes the comparison PNG and draws text labels.	It is already on Kaggle. The code keeps working without it.
psutil (optional)	Finds out when the session started, for the 12-hour budget.	It is already on Kaggle. There is a fallback if it is missing.
Python-only standard library	`threading` , `queue` , `subprocess` , `json` , `hashlib` , `urllib` …	These need no extra installs, which means fewer things can break.
Git + GitHub	Version history, and where the notebook is downloaded from.	It is the standard for sharing code.

Notice what is **missing**: there is **no** `requirements.txt` . The project relies on what Kaggle already has installed. That is convenient, but it is a weakness (see Part 3).

Architecture

The project is a **single-process pipeline**, not a client–server app. You (in the browser) tell the notebook to run. The notebook runs `engine.py` code on Kaggle's computer. That code drives three workers at the same time: an ffmpeg **decoder**, the **GPU model**, and an ffmpeg **encoder**. The results land in Kaggle's Output folder.

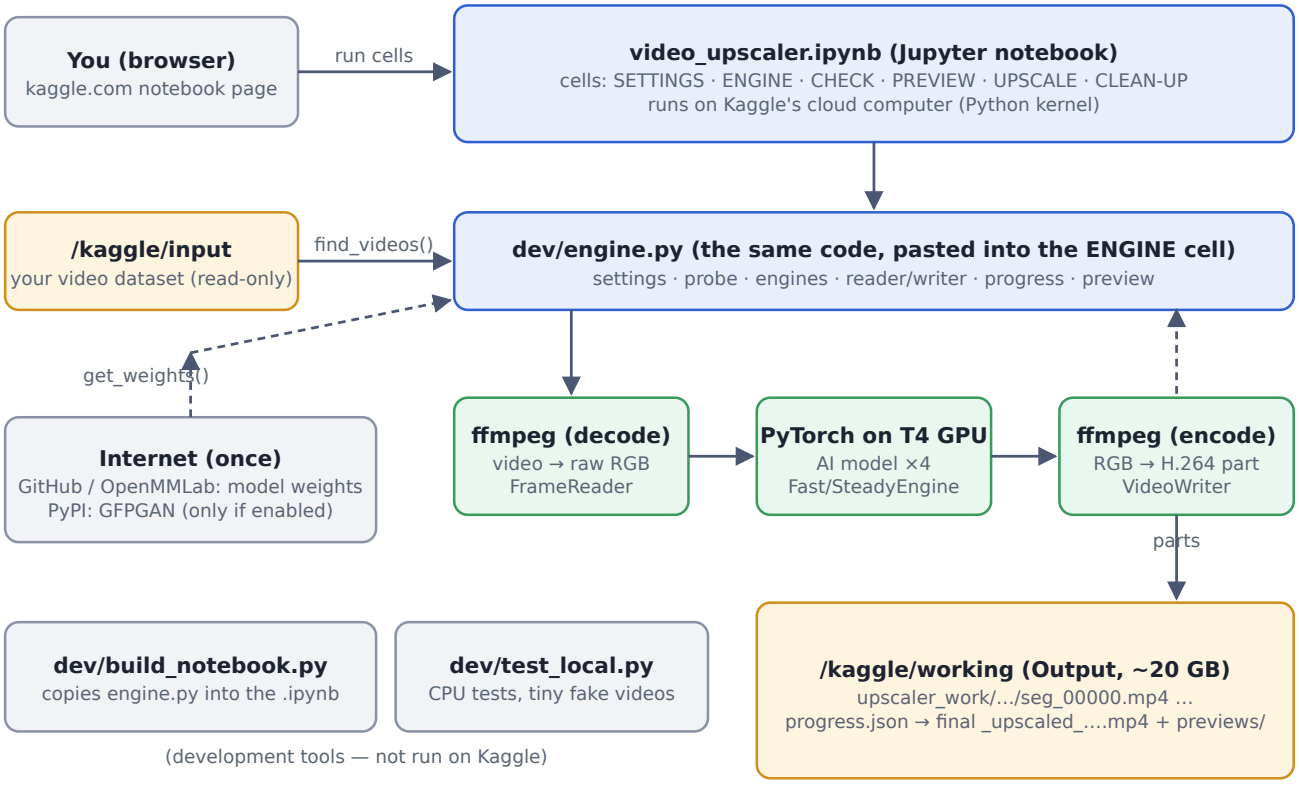


Figure 1 — How the parts connect. Solid arrows: every run. Dashed: only sometimes.

Folder structure choices

- `video_upscaler.ipynb` — the only file a user needs. It is *generated* (built automatically), so do not edit it by hand.
- `dev/engine.py` — the real source code (the "single source of truth"). It is pasted into the ENGINE cell.

- `dev/build_notebook.py` — builds the notebook from `engine.py` plus the text of the other cells.
- `dev/test_local.py` — an automatic test that runs everything on a normal CPU with tiny fake videos.
- `README.md` — the user guide. `.gitignore` — files Git should ignore.

The `dev/` folder means "for developers". It keeps the user-facing part to **one file**, which makes it simple for beginners.

Where is the "database"? There isn't a real one. The project's only saved state is a small **JSON file** (`progress.json`) plus the finished video parts on disk. For a tool that one person runs at a time, that is the right choice: a database would add setup and moving parts without solving any problem.

Journey of a request: "upscale my video"

Here is what happens when you press **Shift + Enter** on the UPSCALE cell, followed all the way to the disk and back. (In a web app, this would be "button click → API → database". Here the equivalent of the "database" is the Output folder.)

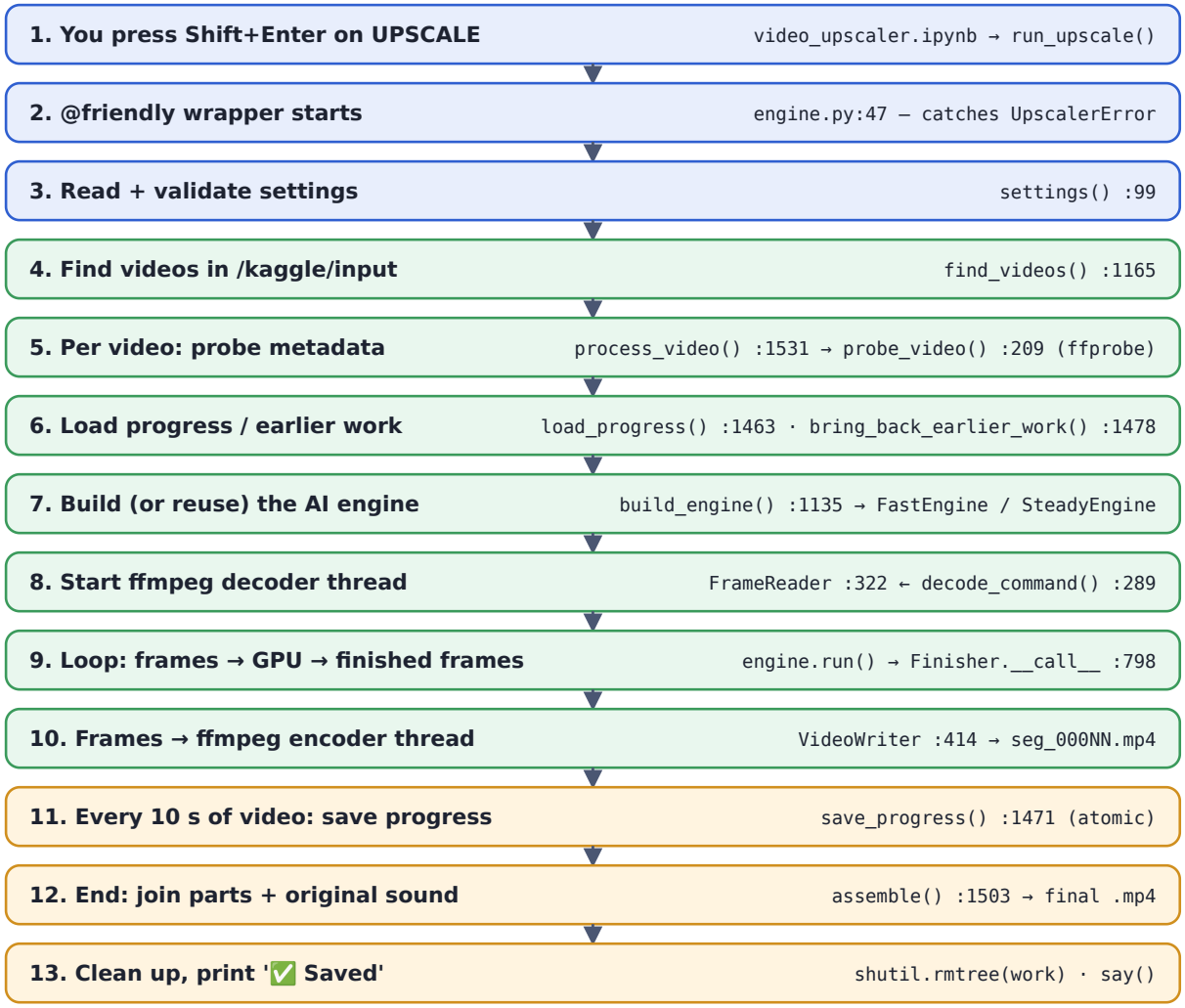


Figure 2 — The journey of one UPSCALE run, with the function and line number of each step.

Step by step, in plain words

1. **The cell runs `run_upscale()`.** It is wrapped by `@friendly`, so any expected error becomes a clean message instead of a scary traceback.

2. **`settings()`** reads your SETTINGS cell and checks every value (Is SCALE between 1 and 4? Is MODEL "fast" or "steady"?).
3. **`find_videos()`** walks through `/kaggle/input` and collects video files. It skips the notebook's own output folders.
4. **`process_video()`** asks **ffprobe** about the video: its size, frame rate, rotation, colour type and whether it has audio.
5. It checks for **saved progress** (`progress.json`). If an earlier Kaggle run's output is attached, it copies that progress back.
6. **`build_engine()`** loads the AI model once, or reuses it if it is already loaded with the same settings.
7. **`FrameReader`** starts ffmpeg, which decodes the video into raw RGB frames. A background thread keeps a queue of frames ready.
8. **The loop:** read a few frames → `engine.run()` upscales them on the GPU → `Finisher` resizes them, blends in a little of the original, and adds grain → each finished frame is handed to `VideoWriter`.
9. **`VideoWriter`** feeds the frames to a second ffmpeg process, which compresses them into a short MP4 part (`seg_00000.mp4`, `seg_00001.mp4`, …).
10. After **every 10 seconds of video**, `save_progress()` writes `progress.json` **atomically** (it can never be half-written).
11. When the video ends, **`assemble()`** joins all the parts and copies in the **original audio**, giving the final MP4.
12. The temporary parts are deleted and the result is printed. You download it from **Output**.

The same journey with a **preview** uses `run_preview()`. It reads only a short clip, writes two videos (upscaled and before/after) and one PNG image, then measures the speed to estimate the full run time.

What is NOT in this project (and how to answer if asked)

Interviewers often ask about web, mobile and database topics. Your project has none of them, and that is a correct design choice for a personal tool that runs in a notebook. Here is how you could answer:

Topic you may be asked about	In your project?	A good honest answer
Frontend / mobile UI (React, RecyclerView, ViewModel…)	No	"The interface is the notebook itself, plus a settings cell. For a personal tool, a UI would have added cost and nothing else."
Backend server, routing, middleware, controllers	No	"It runs as one Python process. The closest thing to a 'controller' is the top-level functions like `run_upscale`, which coordinate the services underneath."
Authentication (JWT, sessions), password hashing, CORS, rate limiting	No	"There are no users or network API. Kaggle's own login protects the notebook. If I turned it into a web service, I would add auth and rate limiting first."
Database, ORM, migrations, SQL injection	No	"State is a JSON checkpoint file with atomic writes, which was enough for one user. I do use the same idea a database uses: never leave half-written data."

Topic you may be asked about	In your project?	A good honest answer
`.env` files / secrets	No	"There are no secrets. All model downloads are public."
CI/CD, Docker, deployment	No	"'Deployment' is importing the notebook into Kaggle. Kaggle provides the container."

Part 2A — Interview favourites (★★★)

These twelve concepts are the ones most worth practising out loud. They show up in interviews for almost any software job, not only AI or video jobs.

1. Streaming pipeline with subprocesses and pipes

★★★ Interview favourite · Category: performance & architecture

What it is

Analogy: picture a sushi conveyor belt. The chef doesn't cook all 500 plates first, stack them on the floor, and then serve them. Each plate goes onto the belt as soon as it is ready, and customers take plates while the chef keeps cooking.

Technically: a *streaming pipeline* processes data piece by piece while it flows, instead of saving everything to disk between steps. Your code starts **ffmpeg** as a separate program (a *subprocess*) and connects to it with a **pipe**, a direct data channel between two programs. ffmpeg writes raw frames into the pipe, Python reads them, the GPU upscales them, and Python writes the results into a *second* pipe to another ffmpeg, which compresses them into MP4.

Where it is in your project

dev/engine.py · lines 322–337 · FrameReader starts the decoder with stdout=PIPE

```
322 class FrameReader:
323     """Reads pictures from ffmpeg in the background so the GPU never waits."""
324
325     def __init__(self, cmd, width, height, log_path, prefetch=24):
326         self.w, self.h = width, height
327         self.frame_bytes = width * height * 3
328         self.log_path = log_path
329         self.log = open(log_path, "ab")
330         self.proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=self.log, bufsize=0)
331         self.q = queue.Queue(maxsize=prefetch)
332         self.pending = deque()
333         self.eof = False
334         self.stop = False
335         self.frames_read = 0
336         self.thread = threading.Thread(target=self._pump, daemon=True)
337         self.thread.start()
```

dev/engine.py · lines 421–429 · VideoWriter starts the encoder and writes to its stdin

```
421         cmd = ["ffmpeg", "-hide_banner", "-nostdin", "-loglevel", "error", "-y",
422                "-f", "rawvideo", "-pix_fmt", "rgb24", "-s", f"{width}x{height}",
423                "-framerate", fps_str, "-i", "pipe:0",
424                "-vf",
"scale=out_color_matrix=bt709:out_range=tv:flags=bicubic+accurate_rnd+full_chroma_int,format=yuv420p",
425                "-c:v", "libx264", "-preset", S.ENCODER_SPEED, "-crf", str(S.VIDEO_QUALITY), "-tune",
tune,
426                "-colorspace", "bt709", "-color_primaries", "bt709", "-color_trc", "bt709", "-
color_range", "tv",
427                "-movflags", "+faststart", path]
428         self.log = open(log_path, "ab")
429         self.proc = subprocess.Popen(cmd, stdin=subprocess.PIPE, stderr=self.log, bufsize=0)
```

dev/engine.py · lines 307–307 · decode_command: "send raw RGB to pipe:1 (stdout)"

```
307     cmd += ["-vf", ",".join(vf)] + passthrough_flag() + ["-f", "rawvideo", "-pix_fmt", "rgb24",
"pipe:1"]
```

Why we used it

Your original plan was "split the video into image files, upscale them, rebuild the video". A 10-minute 1080p video is about 18,000 frames. As PNG files that could easily be **50–100 GB**, far more than Kaggle's ~20 GB output limit, and writing and reading all those files is slow. Streaming keeps only a handful of frames in memory at any moment, so the disk never fills up with frames.

How it works

1. `subprocess.Popen(cmd, stdout=subprocess.PIPE, ...)` starts ffmpeg and gives Python the reading end of its output.
2. `decode_command` tells ffmpeg to output `-f rawvideo -pix_fmt rgb24 pipe:1`. That means "no file format, just the raw bytes of each picture, 3 bytes per pixel, into standard output".
3. Because every frame has exactly `width × height × 3` bytes, Python knows where one frame ends: it reads exactly `frame_bytes` bytes (`_read_exact`) and turns them into a NumPy array.
4. On the other side, `VideoWriter` starts ffmpeg with `-i pipe:0` (read from standard input) and writes each finished frame's bytes into `proc.stdin`.
5. `stderr=self.log` sends ffmpeg's error messages to a log file. That way they don't clog a pipe nobody reads, and you can show them when something fails.

What would go wrong without it

- **The disk would fill up:** Kaggle stops at about 20 GB, so a long video would crash partway through.
- **It would be slower**, because PNG encoding and decoding of thousands of large images costs a lot of CPU time.
- You would need extra "clean up the temporary frames" logic, which can itself fail and leave junk behind.

Alternatives and trade-offs

Option	Pros	Cons
Temporary image files (your original plan)	Easy to understand and inspect.	Huge disk use, slow, cleanup needed.
A Python video library (PyAV, imageio, OpenCV `VideoCapture`)	No raw byte handling.	It is another dependency, and colour handling and seeking are less controllable. OpenCV can't write H.264 with good settings.
Pipes to ffmpeg (chosen)	Uses the best video tool, has zero disk use for frames, and the work runs in parallel.	You must handle bytes, process lifetimes and errors yourself.

Interview prep

> **Q: Why didn't you just save frames as images?**
>
> A: The disk on Kaggle is limited to about 20 GB, and one long video as PNGs can be bigger than that. So I stream the frames: ffmpeg decodes into a pipe, the GPU processes them, and a second ffmpeg encodes from a pipe. Only a few frames are ever in memory, and nothing piles up on disk.
>
> **Q: How does Python know where one frame ends in a stream of bytes?**
>
> A: I ask ffmpeg for raw RGB, so every frame is exactly width × height × 3 bytes. I read that exact number of bytes each time. Fixed-size records mean I don't need any separator.
>
> **Q: What happens to ffmpeg's error messages?**
>
> A: They go to a log file instead of a pipe. If nobody read the error pipe, it could fill up and freeze ffmpeg, which is a classic deadlock. When something fails, I show the last part of the log.

Future use

Use this whenever you process **large data that doesn't fit on disk or in memory**: log files, sensor data, big CSV files, audio, or calling command-line tools such as ImageMagick or git from code. Think "conveyor belt, not warehouse".

2. Producer–consumer with threads and bounded queues

★★★ Interview favourite · Category: concurrency

What it is

Analogy: a bakery. One person bakes bread (the *producer*) and puts loaves on a shelf. Another person sells them (the *consumer*). The shelf has room for only 24 loaves. If it's full, the baker waits, and if it's empty, the seller waits. Neither one has to work at the other's speed every second.

Technically: a **thread** is a second line of work inside the same program. A **queue** is a thread-safe shelf between them: `put()` adds an item and `get()` takes one. A **bounded** queue (`maxsize=...`) makes a fast producer wait when the shelf is full. This is called **backpressure**, and it stops memory from overflowing.

Where it is in your project

dev/engine.py · lines 350–370 · _pump (producer thread) and read (consumer side)

```
350     def _pump(self):
351         try:
352             while not self.stop:
353                 buf = self._read_exact()
354                 if buf is None:
355                     break
356                 self.q.put(np.frombuffer(buf, np.uint8).reshape(self.h, self.w, 3))
357         finally:
358             self.q.put(None)
359 
360     def read(self):
361         if self.pending:
362             return self.pending.popleft()
363         if self.eof:
364             return None
365         frame = self.q.get()
366         if frame is None:
367             self.eof = True
368             return None
369         self.frames_read += 1
370         return frame
```

dev/engine.py · lines 436–453 · VideoWriter: main thread produces, writer thread consumes

```
436     def _pump(self):
437         while True:
438             frame = self.q.get()
439             if frame is None:
440                 break
441             if self.error:
442                 continue
443             try:
444                 self.proc.stdin.write(memoryview(np.ascontiguousarray(frame)).cast("B"))
445             except Exception as e:  # ffmpeg died (disk full, etc.)
446                 self.error = e
447 
448     def write(self, frame):
449         if self.error:
450             raise UpscalerError("The video encoder (ffmpeg) stopped unexpectedly. Is the disk full?\n"
451                                 + log_tail(self.log_path))
452         self.q.put(frame)
453         self.count += 1
```

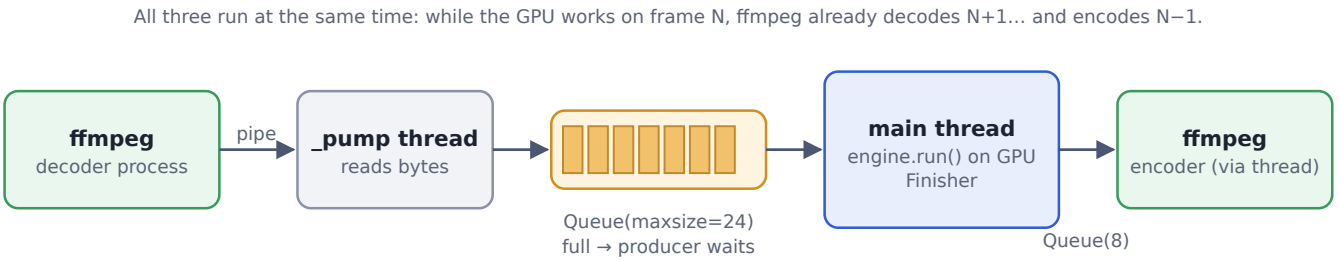


Figure 3 — Producer–consumer pipeline with two bounded queues.

Why we used it

There are three slow jobs: **decoding** (CPU, ffmpeg), **upscaling** (GPU) and **encoding** (CPU, ffmpeg). Done one after another, the GPU would sit idle while ffmpeg works, and the reverse. With threads and queues, all three run **at the same time**. While the GPU upscales frame 100, ffmpeg is already decoding frame 101 and encoding frame 99.

How it works

1. `FrameReader.__init__` creates `queue.Queue(maxsize=24)` and starts `threading.Thread(target=self._pump, daemon=True)`.
2. `_pump` loops: read one frame of bytes, then `self.q.put(frame)`. If 24 frames are already waiting, `put` simply **waits**.
3. At the end of the video, the `finally:` block puts `None` in the queue. This is a **sentinel**, an agreed "that's all" signal.
4. The main thread calls `read()`, which calls `self.q.get()`. If it gets `None`, it marks `eof` (end of file).
5. `VideoWriter` is the same idea reversed. The main thread `put`s finished frames into `Queue(maxsize=8)`, and a writer thread pushes them into ffmpeg's input.
6. `daemon=True` means "don't keep Python alive just for this thread", so a crash can't leave the notebook hanging forever.
7. Errors in the writer thread are stored in `self.error` and raised later on the main thread. An exception inside a thread would otherwise vanish silently.

What would go wrong without it

- **No queue limit:** a fast decoder could pile up thousands of 4K frames in RAM and crash the session (about 25 MB per 4K frame).
- **No threads:** each step would wait for the others, making the whole run noticeably slower and wasting paid-for (well, free-but-limited) GPU hours.
- **No sentinel:** the consumer wouldn't know whether the producer is just slow or finished, and might wait forever.

Alternatives and trade-offs

- **`asyncio`** (async/await) is great for waiting on *network* calls, but here the work is blocking pipe reads plus heavy GPU calls, which threads handle more simply.
- **`multiprocessing`** gets around Python's GIL (Global Interpreter Lock), but frames would have to be copied between processes. That's unnecessary here: the threads spend their time waiting on

I/O, and while they wait Python releases the GIL.

- **Threads + `queue.Queue` (chosen)** is in the standard library, simple, and well understood.

Interview prep

> **Q: What is backpressure and where do you use it?**
>
> A: Backpressure means a fast producer is slowed down so it can't overwhelm a slow consumer. My decoder puts frames into a queue with maxsize 24. When the GPU is slower, the queue fills up and the decoder automatically waits, so memory stays flat.
>
> **Q: Python has the GIL. Do your threads actually help?**
>
> A: Yes. The GIL only blocks running Python code at the same time. My threads mostly wait on pipe I/O, and PyTorch's GPU work releases the GIL too. So decoding, GPU work and encoding really do overlap.
>
> **Q: How do you stop a producer thread cleanly?**
>
> A: With a sentinel value, None, at the end of the stream, plus a stop flag. In `close()` I kill ffmpeg and drain the queue so the thread isn't stuck waiting on a full shelf.

Future use

Use this for **web scrapers** (download threads → parse thread), **image or upload processing**, **logging** (the app produces logs, a background thread writes them), or any job where one step is fast and another is slow.

3. Checkpointing and resume

★★★ Interview favourite · Category: reliability

What it is

Analogy: a video game save point. If you die, you start again from the last save point, not from the beginning of the game.

Technically: checkpointing means regularly saving enough information about your progress that, after a crash or stop, the program can **resume** from the last checkpoint. Your project saves the video in short parts ("segments") and keeps a small file, `progress.json`, listing which parts are finished.

Where it is in your project

dev/engine.py · lines 1555–1570 · load progress, keep only parts that really exist, delete half-written ones

```
1555     ppath = os.path.join(work, "progress.json")
1556     progress = load_progress(ppath) or dict(video=info["name"], settings=settings_label(S),
frames_done=0,
1557                                             segments=[], complete=False)
1558     # forget any part whose file went missing; delete half-written parts
1559     kept, frames = [], 0
1560     for s in progress["segments"]:
1561         if os.path.exists(os.path.join(work, s["file"])):
1562             kept.append(s)
1563             frames += s["frames"]
1564         else:
1565             break
1566     progress["segments"], progress["frames_done"] = kept, frames
1567     known = {s["file"] for s in kept}
1568     for f in glob.glob(os.path.join(work, "seg_*.mp4")):
1569         if os.path.basename(f) not in known:
1570             os.remove(f)
```

dev/engine.py · lines 1629–1632 · after each finished part: record it and save

```
1629                 writer.close()
1630                 progress["segments"].append(dict(file=seg_name, frames=got))
1631                 progress["frames_done"] += got
1632                 save_progress(ppath, progress)
```

dev/engine.py · lines 1582–1587 · restart the decoder exactly where we stopped

```
1582         rstart = max(0, start - ctx)
1583         if rstart > 0:
1584             print("   Skipping ahead to where we stopped (can take a few minutes for long videos)
..."")
1585         log = os.path.join(work, "ffmpeg.log")
1586         reader = FrameReader(decode_command(info, start_frame=rstart), info["width"], info["height"],
log)
1587         before = reader.read_n(start - rstart)
```

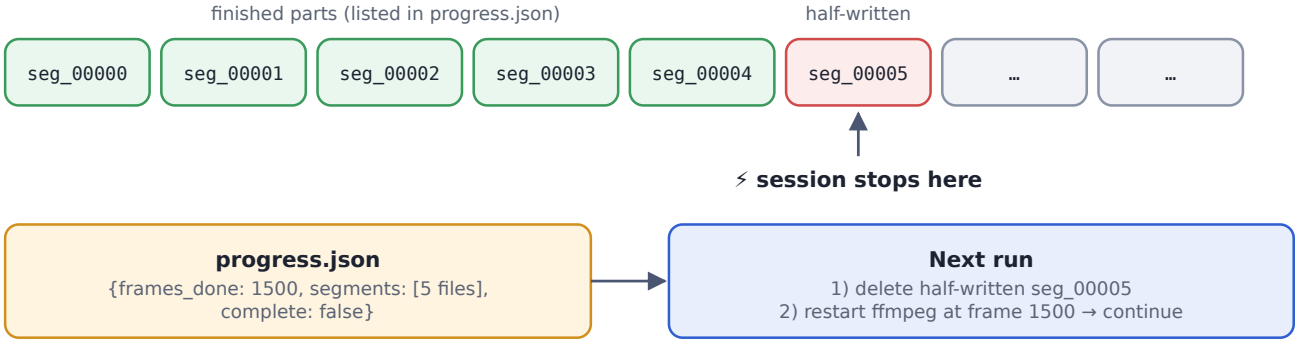


Figure 4 — Checkpointing: finished parts survive; the half-written one is thrown away.

Why we used it

Kaggle stops every session after **12 hours**, and sessions can also die if the browser is closed or the internet drops. A long video at high quality can take longer than 12 hours. Without checkpoints, you would lose *all* the work and could never finish such a video.

How it works

1. `SAVE_EVERY_SECONDS = 10` means one segment holds 10 seconds of video (for example, 300 frames at 30 fps).
2. Each segment is encoded into its own file, `seg_00012.mp4`.
3. **Only after** a segment's encoder closes successfully does the code append it to `progress["segments"]` and call `save_progress`.
4. On the next run, `load_progress` reads the file. Parts whose file is missing are dropped, and **any `seg_*.mp4` not listed is deleted**, because it must be half-written from the crash.
5. `decode_command(info, start_frame=rstart)` uses ffmpeg's `select=gte(n, START)` filter to start at **exactly** the right frame. A test proves this is frame-exact.
6. When everything is finished, `assemble()` joins the parts and the work folder is deleted.
7. **Across Kaggle sessions:** `bring_back_earlier_work()` looks in `/kaggle/input` for the progress folder of a previous run (attached as input) and copies it back.

What would go wrong without it

Every interruption would mean starting from zero. A 13-hour job would **never** finish on Kaggle. A half-written part could also be glued into the final video and corrupt it, which is why unlisted parts are deleted.

Alternatives and trade-offs

- **Bigger segments** mean less overhead but more lost work per crash. **Smaller** ones mean more files. 10 seconds is a balance.
- **Saving only a frame counter**, without separate part files, would require writing into one growing MP4, which can't be safely "continued" after a crash.
- A **database** could store progress, but it's overkill for one JSON object. The important property, never trusting half-finished work, is the same.
- *Known trade-off:* to resume at frame N, ffmpeg still has to **decode** (but not upscale) frames 0…N−1, which can take a few minutes for very long videos.

Interview prep

> **Q: How does your program survive the 12-hour limit?**
>
> A: It writes the output in 10-second segments and records each finished segment in a JSON file. On restart it deletes any half-written segment, seeks the decoder to the exact next frame, and continues. At the end it concatenates the segments and adds the audio.
>
> **Q: How did you prove the resumed video is correct?**
>
> A: My test stops a run partway on purpose, then resumes it. It also runs the same video in one go, and checks that the two results are byte-for-byte identical frames.
>
> **Q: What's the order of operations, and why does it matter?**
>
> A: First finish and close the segment file, then record it in progress.json. If I recorded it first and then crashed, the checkpoint would point at a broken file.

Future use

Use checkpointing for **long batch jobs** of any kind: data migrations, training ML models (saving model checkpoints), bulk emailing, scraping thousands of pages, or file uploads that can be resumed.

4. Atomic writes (write-to-temp, then rename)

★★★ Interview favourite · Category: reliability / data safety

What it is

Analogy: you're rewriting a shopping list. Instead of erasing the old list and writing the new one (if someone interrupts you halfway, there's a half-erased list), you write the new list on a fresh piece of paper and then swap it in one quick move.

Technically: an **atomic** operation happens completely or not at all, with no in-between state. Writing a file is *not* atomic: a crash halfway leaves a corrupt file. **Renaming** a file on the same disk *is* atomic. So the trick is: write to `progress.json.tmp`, then `os.replace(tmp, real_name)`.

Where it is in your project

dev/engine.py · lines 1471–1475 · save_progress

```
1471 def save_progress(path, data):
1472     tmp = path + ".tmp"
1473     with open(tmp, "w") as f:
1474         json.dump(data, f, indent=1)
1475     os.replace(tmp, path)
```

dev/engine.py · lines 521–527 · downloads go to .part first, then are renamed

```
521                 tmp = dest + ".part"
522                 with urllib.request.urlopen(req, timeout=60) as r, open(tmp, "wb") as f:
523                     shutil.copyfileobj(r, f, 1024 * 1024)
524                 if os.path.getsize(tmp) < 100_000:
525                     raise IOError("downloaded file is too small (probably an error page)")
526                 os.replace(tmp, dest)
527                 return dest
```

dev/engine.py · lines 1509–1509 · the final video is written as .partial.mp4 first

```
1509     tmp = final_path + ".partial.mp4"
```

dev/engine.py · lines 1524–1524 · …and renamed only after ffmpeg succeeded

```
1524             os.replace(tmp, final_path)
```

Why we used it

The progress file is the most important file in the project: if it's corrupt, the resume breaks. Kaggle can stop a session at **any moment**, including halfway through writing it. The same logic applies to model downloads and the final video. A half-downloaded model file, or a half-joined video that *looks* finished, would be a nasty, confusing bug.

How it works

1. `tmp = path + ".tmp"` builds a temporary name next to the real file. It must be on the same disk.
2. `json.dump(data, f)` writes the whole content to the temporary file.
3. `os.replace(tmp, path)` swaps it in. The operating system guarantees that anyone reading `path` sees either the complete old file or the complete new one.
4. Downloads use the same pattern (`dest + ".part"`) plus a **size check**, so a 5 KB error page saved as a "model" is rejected.

5. `process_video` also checks `if os.path.exists(final_path)` to skip finished videos. That check is only safe *because* the final file appears atomically.

What would go wrong without it

After a crash during writing: a corrupt `progress.json` (resume fails, or work is lost), a truncated model file (confusing "unexpected EOF" errors forever, because the code thinks it's already downloaded), or a broken final MP4 that's skipped next time because "it already exists".

Alternatives and trade-offs

- **Databases** (SQLite, PostgreSQL) give you this with **transactions**: a group of changes that all succeed or all fail. That's the same idea at a bigger scale.
- **Write-ahead logs / journaling** are how databases do it internally.
- For one small JSON file, temp file + rename is the simplest correct solution. (To be extra safe against power loss, you can also call `os.fsync` before renaming. Your code doesn't, which is acceptable on Kaggle.)

Interview prep

> **Q: How do you make sure a file is never half-written?**
>
> A: I write to a temporary file in the same folder, then `os.replace` it onto the real name. A rename is atomic, so readers always see either the old or the new complete file.
>
> **Q: Where else does this idea appear?**
>
> A: Database transactions follow the same all-or-nothing rule. So do package managers installing updates, and editors saving files. In my project I use it for the progress file, model downloads and the final video.

Future use

Use it any time you save **config files, caches, progress or state**, or download files. If a crash at the worst moment would leave a broken file, use temp + rename.

5. Graceful degradation and automatic fallbacks

★★★ Interview favourite · Category: reliability / error handling

What it is

Analogy: your car's sat-nav. If the motorway is closed, it doesn't give up. It tries the A-road, then smaller roads, and only says "no route" when nothing works.

Technically: graceful degradation means that when the best way fails, the program automatically tries a slightly worse but still-working way, instead of crashing. Your project has several of these **fallback ladders**.

Where it is in your project

dev/engine.py · lines 882–912 · FastEngine: out of memory → smaller batch → tiles; bad fp16 → fp32

```
882     def _shrink(self):
883         if self.batch > 1:
884             self.batch //= 2
885             return
886         smaller = [t for t in TILE_STEPS if self.tile is None or t < self.tile]
887         if not smaller:
888             raise UpscalerError("Ran out of GPU memory even with the smallest tiles. "
889                                 "Try a smaller SCALE or a lower-resolution video.")
890         self.tile = smaller[0]
891         print(f"\n(GPU memory is tight - switching to tiles of {self.tile} px. This is automatic.)")
892 
893     def run(self, before, core, after):
894         out, i = [], 0
895         while i < len(core):
896             chunk = core[i:i + self.batch]
897             try:
898                 out.extend(self._process(chunk))
899                 i += len(chunk)
900             except NonFiniteOutput:
901                 if not self.half:
902                     raise UpscalerError("The model produced broken pixels. Please report this video.")
903                 print("\n(Half precision gave bad pixels - switching to full precision. Automatic.)")
904                 self.half = False
905             except Exception as e:
906                 if not is_memory_error(e):
907                     raise
908                 free_memory()
909                 self._shrink()
910             if self.tile:
911                 free_memory()
912         return out
```

dev/engine.py · lines 1512–1526 · assemble: copy audio → re-encode audio → video without audio

```
1512     attempts = []
1513     if info["has_audio"]:
1514         audio_in = ["-i", info["path"], "-map", "0:v:0", "-map", "1:a?", "-map_metadata", "1"]
1515         attempts.append(audio_in + ["-c:v", "copy", "-c:a", "copy"])
1516         attempts.append(audio_in + ["-c:v", "copy", "-c:a", "aac", "-b:a", "256k"])
1517     attempts.append(["-map", "0:v:0", "-c:v", "copy"])
1518     err = ""
1519     for i, extra in enumerate(attempts):
1520         r = subprocess.run(base + extra + ["-movflags", "+faststart", tmp], capture_output=True,
text=True)
1521         if r.returncode == 0 and os.path.exists(tmp) and os.path.getsize(tmp) > 0:
1522             if info["has_audio"] and i == len(attempts) - 1:
1523                 print("⚠️  The sound could not be added - the video was saved WITHOUT sound.")
1524             os.replace(tmp, final_path)
1525             return
1526         err = r.stderr
```

dev/engine.py · lines 765–771 · deciding whether an error is a memory error

```
765 def is_memory_error(e):
766     if isinstance(e, getattr(torch.cuda, "OutOfMemoryError", ())):
767         return True
768     msg = str(e)
769     return any(s in msg for s in ("out of memory", "CUDNN_STATUS", "CUBLAS_STATUS_ALLOC",
770                                   "canUse32BitIndexMath", "cuDNN error", "valid cuDNN algorithm",
771                                   "DefaultCPUAllocator: can't allocate"))
```

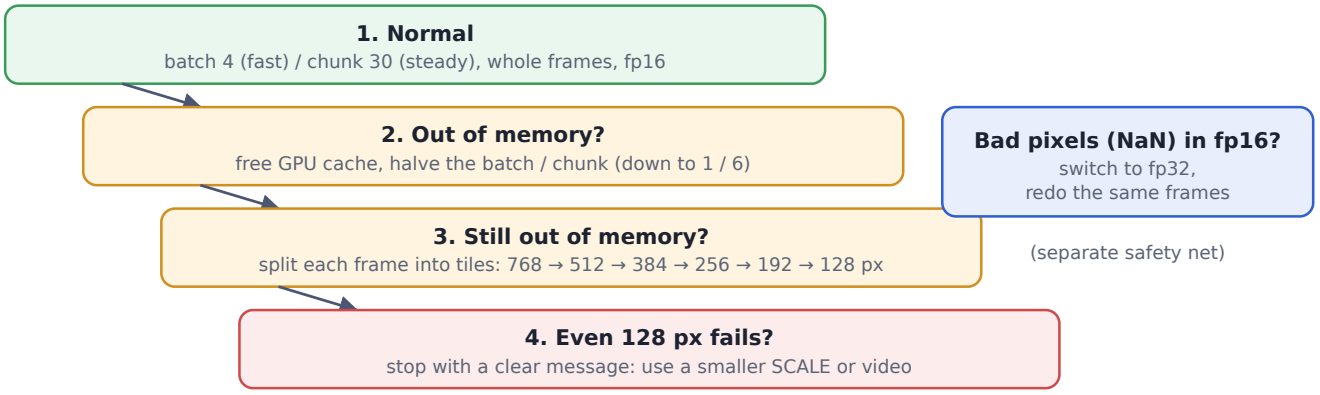


Figure 7 — The automatic fallback ladder when GPU memory runs out.

Why we used it

A T4 GPU has 16 GB of memory. A large video, a large SCALE, or the memory-hungry "steady" model can run out of it. A beginner shouldn't have to understand "CUDA out of memory" and tune batch sizes by hand. Likewise, some audio formats can't be copied into an MP4 file, and older or newer ffmpeg versions use different option names.

How it works

1. `run()` tries `_process(chunk)`.
2. If an exception happens, `is_memory_error(e)` checks for PyTorch's `OutOfMemoryError` or well-known out-of-memory messages from cuDNN/cuBLAS.
3. If it isn't a memory error, it is re-raised (`raise`), because hiding real bugs would be bad.
4. If it is, `free_memory()` clears the GPU cache, then `_shrink()` halves the batch (4 → 2 → 1). After that it switches to **tiles** of 768, 512 … 128 pixels.
5. The `while i < len(core)` loop then **retries the same frames**, so no frame is skipped.

6. If even the smallest tile fails, it raises an `UpscalerError` with advice: lower SCALE.
7. `NonFiniteOutput` (NaN, meaning "not a number", or infinite pixels from half precision) switches to full precision and retries.
8. `assemble` tries three ffmpeg commands in order and warns you if the sound had to be dropped.
9. `passthrough_flag()` picks `-fps_mode` for ffmpeg 5+ or `-vsync` for older versions.

What would go wrong without it

The notebook would crash with long technical errors on bigger videos. Users would give up, or have to edit advanced settings. And a single unusual audio format would block the whole video from being saved.

Alternatives and trade-offs

- **Fail fast** with a clear message and let the user lower settings. It's simpler and more predictable, but a worse experience for beginners.
- **Check memory beforehand** (estimating memory use). That's hard to get right across models and resolutions, so "try, and shrink if needed" is more robust.
- *Trade-off:* automatic fallbacks can **hide** problems. That's why your code prints a short note ("GPU memory is tight – switching to tiles") and only catches *known* error types.

Interview prep

> **Q: What happens if the GPU runs out of memory?**
>
> A: The code catches only out-of-memory errors, frees the GPU cache, then retries the same frames with a smaller batch. If that's still too big, it splits each frame into overlapping tiles. It tells the user what it did, and only stops with advice if even the smallest tile fails.
>
> **Q: Isn't catching exceptions like that dangerous?**
>
> A: It would be if I caught everything. I check that it really is a memory error and re-raise anything else, so real bugs still surface.

Future use

Use it for **calling external APIs** (fall back to a cache or a backup provider), **image or video processing**, **mobile apps with weak network** (offline mode), and **feature detection** across tool or browser versions.

6. Retry with exponential backoff

★★★ Interview favourite · Category: networking / reliability

What it is

Analogy: calling a friend who doesn't pick up. You try again after 1 minute, then 2, then 4. You don't call 100 times a second.

Technically: network calls fail sometimes for temporary reasons (a busy server, a Wi-Fi blip). **Retrying** handles that. **Exponential backoff** means waiting longer after each failure (1 s, 2 s, 4 s, 8 s), so you don't hammer a struggling server and you give the problem time to clear.

Where it is in your project

dev/engine.py · lines 515–536 · get_weights: up to 4 attempts per URL, waiting 2^attempt seconds

```
515     last_err = None
516     for url in urls:
517         for attempt in range(4):
518             try:
519                 print(f"Downloading {fname} ...")
520                 req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
521                 tmp = dest + ".part"
522                 with urllib.request.urlopen(req, timeout=60) as r, open(tmp, "wb") as f:
523                     shutil.copyfileobj(r, f, 1024 * 1024)
524                 if os.path.getsize(tmp) < 100_000:
525                     raise IOError("downloaded file is too small (probably an error page)")
526                 os.replace(tmp, dest)
527                 return dest
528             except Exception as e:
529                 last_err = e
530                 time.sleep(2 ** attempt)
531     hint = ("Turn on Internet: in the notebook's right-hand panel open 'Session options' "
532             "(or 'Settings') and switch 'Internet' ON. Your Kaggle account must be phone-verified for
this.")
533     if key == "realbasicvsr":
534         hint += ("\nOr download the RealBasicVSR .pth file yourself, upload it to Kaggle as a dataset,
and "
535                  "put its path in STEADY_WEIGHTS_FILE. Or simply use MODEL = \"fast\".")
536     raise UpscalerError(f"Could not download the model file {fname}.\nReason: {last_err}\n{hint}")
```

Why we used it

Model files are downloaded from GitHub and OpenMMLab when the notebook starts. If one download hiccups, the whole run would fail, which is annoying on a free GPU quota where sessions take time to start.

How it works

1. `for url in urls:` there can be several mirror URLs, which are tried in order.
2. `for attempt in range(4):` makes up to 4 tries each.
3. Inside `try`: download to `.part`, check the size, rename. On success, `return dest`.
4. On any exception, it remembers the error (`last_err`) and waits `time.sleep(2 ** attempt)`: 1, 2, 4, then 8 seconds.
5. If everything fails, it raises one friendly `UpscalerError` that includes the real reason and tells the user to turn on Internet in Kaggle.
6. `timeout=60` makes sure a dead connection can't hang forever.

What would go wrong without it

Random one-off network errors would stop the notebook. Without a timeout, a stuck connection could freeze the cell with no message at all.

Alternatives and trade-offs

- Libraries like `tenacity` or `requests` + `urllib3.Retry` do this with more options (for example only retrying certain errors, or adding *jitter*, a small random extra wait so many clients don't retry at the same instant).
- *Honest weakness:* your loop retries **every** error, even ones that will never succeed (like "404 file not found"). It also doesn't add jitter. Both are fine for a personal tool, but worth mentioning.

Interview prep

> **Q: Why exponential and not a fixed 1-second wait?**
> A: If the server is overloaded, retrying quickly makes it worse. Doubling the wait gives it time to recover and spreads out the load. In bigger systems I'd add random jitter so thousands of clients don't retry in sync.
>
> **Q: What would you improve in your retry code?**
> A: I'd only retry errors that can be temporary, like timeouts and 5xx server errors, not 404s. I'd add jitter, and verify the file with a checksum.

Future use

Use it with **any HTTP API call**, payment providers, email sending, cloud storage uploads, database reconnects, and message queues.

7. Input validation

★★★ Interview favourite · Category: robustness / security basics

What it is

Analogy: a bouncer at the door who checks IDs *before* people get in, rather than dealing with trouble inside.

Technically: input validation means checking that data from outside your code (user input, files, APIs) is the right type and within allowed ranges **before** using it, and giving a clear message when it isn't. "Never trust input" is one of the most important rules in software.

Where it is in your project

dev/engine.py · lines 106–142 · settings(): collect all problems, then report them together

```
106     problems = []
107     try:
108         S.SCALE = float(S.SCALE)
109     except Exception:
110         problems.append(f"SCALE must be a number like 2 (you wrote {S.SCALE!r}).")
111     else:
112         if not 1.0 <= S.SCALE <= 4.0:
113             problems.append(f"SCALE must be between 1 and 4 (you wrote {S.SCALE}).")
114     S.MODEL = str(S.MODEL).strip().lower()
115     if S.MODEL not in ("fast", "steady"):
116         problems.append(f'MODEL must be "fast" or "steady" (you wrote {S.MODEL!r}).')
117     for name, lo, hi in [("DENOISE_STRENGTH", 0, 1), ("ORIGINAL_BLEND", 0, 0.9),
118                          ("FILM_GRAIN", 0, 0.2), ("FACE_STRENGTH", 0, 1),
119                          ("VIDEO_QUALITY", 0, 40), ("MAX_RUNTIME_HOURS", 0.1, 100),
120                          ("SAVE_EVERY_SECONDS", 1, 600), ("STEADY_CHUNK_FRAMES", 4, 200),
121                          ("PREVIEW_LENGTH_SECONDS", 1, 60), ("PREVIEW_START_SECONDS", 0, 1e9)]:
122         try:
123             val = float(getattr(S, name))
124         except Exception:
125             problems.append(f"{name} must be a number (you wrote {getattr(S, name)!r}).")
126             continue
127         if not lo <= val <= hi:
128             problems.append(f"{name} must be between {lo} and {hi} (you wrote {val}).")
129         setattr(S, name, val)
130     S.VIDEO_QUALITY = int(round(S.VIDEO_QUALITY)) if isinstance(S.VIDEO_QUALITY, float) else
S.VIDEO_QUALITY
131     S.STEADY_CHUNK_FRAMES = int(S.STEADY_CHUNK_FRAMES) if isinstance(S.STEADY_CHUNK_FRAMES, float)
else S.STEADY_CHUNK_FRAMES
132     S.ENCODER_SPEED = str(S.ENCODER_SPEED).strip().lower()
133     if S.ENCODER_SPEED not in X264_PRESETS:
134         problems.append(f"ENCODER_SPEED must be one of {X264_PRESETS}.")
135     if isinstance(S.ONLY_THESE_FILES, str):
136         S.ONLY_THESE_FILES = [S.ONLY_THESE_FILES] if S.ONLY_THESE_FILES.strip() else []
137     S.FACE_RESTORE = bool(S.FACE_RESTORE)
138     S.USE_HALF_PRECISION = bool(S.USE_HALF_PRECISION)
139     S.SKIP_PREVIEW_IN_SAVE_VERSION = bool(S.SKIP_PREVIEW_IN_SAVE_VERSION)
140     if problems:
141         raise UpscalerError("There is a problem in the SETTINGS cell:\n" + "\n".join("• " + p for p in
problems)
142                             + "\nFix it, run the SETTINGS cell again, then re-run this cell.")
```

Why we used it

The users are beginners editing a SETTINGS cell. Typos like `MODEL = "Fast "` or `SCALE = "2x"` are guaranteed to happen. Without validation they would cause confusing errors **hours later** in the middle of a run, or silently produce wrong results.

How it works

1. Every setting is read with a default (`g.get(k, v)`), so a deleted line doesn't crash the code.
2. Values are **normalised**: `str(S.MODEL).strip().lower()` turns `" Fast "` into `"fast"`.
3. Numbers are converted with `float(...)` inside `try/except`. If that fails, a problem is recorded instead of crashing.
4. A table of `(name, lowest, highest)` checks the ranges in one loop. This is *data-driven* validation, with no repeated `if` statements.
5. All problems are **collected in a list** and reported **together**, so the user can fix them all at once instead of one per run.
6. **Soft warnings** (a high blend or strong grain) are printed but allowed, because they're unusual but not wrong.
7. Other inputs are validated too: `probe_video` checks the file exists and has a video stream, and `find_videos` explains what *is* in the folder when nothing is found.

What would go wrong without it

A bad value could cause a crash deep inside PyTorch with a scary message, waste hours of GPU time, or produce an unusable video (an invalid encoder preset, an impossible scale).

Alternatives and trade-offs

- **Pydantic** or **dataclasses with type hints** give automatic type conversion and errors. They're very popular in web backends (FastAPI).
- **JSON Schema** works for config files.
- Your hand-written approach needs no dependencies and has beginner-friendly messages, but it's longer and easier to forget a field in.

Interview prep

> **Q: How do you handle bad user input?**
>
> A: I validate all settings in one place before any work starts. I normalise text, convert numbers safely, and check ranges from a table. All problems are reported together with the fix. Unusual-but-valid values only get a warning.
>
> **Q: Validate early or late?**
>
> A: Early, at the boundary where the data enters the program. That way bad data never reaches the expensive or dangerous parts. Here it saves hours of wasted GPU time.

Future use

Validate **form fields, API request bodies, URL parameters, config files and uploaded files**. In web apps, validation also protects against attacks such as SQL injection, so always check on the server too, not only in the browser.

8. Custom exceptions and a friendly error strategy

★★★ Interview favourite · Category: error handling

What it is

Analogy: a hospital triage desk. "Expected" problems, like a sprained ankle, get a simple instruction sheet. Unexpected ones get sent to the specialist with all the details.

Technically: an **exception** is Python's way of saying "something went wrong, stop and jump to the nearest handler". You can create **your own exception types** (classes) to mark *expected, explainable* problems, and handle them differently from real bugs.

Where it is in your project

dev/engine.py · lines 34–35 · UpscalerError: "an error with a plain-English explanation"

```
34 class UpscalerError(Exception):
35     """An error with a plain-English explanation (no scary traceback needed)."""
```

dev/engine.py · lines 761–762 · NonFiniteOutput: an internal signal, never shown to the user

```
761 class NonFiniteOutput(Exception):
762     pass
```

dev/engine.py · lines 1125–1129 · an example of a friendly error with the exact fix

```
1125     raise UpscalerError(
1126         "No GPU found. The GPU is switched off.\n"
1127         "Fix: in the right-hand panel of the notebook open 'Session options' (or 'Settings'),\n"
1128         "set ACCELERATOR to 'GPU T4 x2', confirm, then run all cells again from the top.\n"
1129         "(If you don't see the option, verify your phone number in your Kaggle account settings.)")
```

Why we used it

Your user is a beginner. A 40-line traceback about `CUDA` or `NoneType` means nothing to them. But "No GPU found → open Session options → choose GPU T4 x2" is something they can act on. At the same time, real programming bugs should still show the full traceback, so they can be reported and fixed.

How it works

1. `class UpscalerError(Exception)` is a new type that behaves like any exception.
2. Wherever the code detects a problem the *user* can fix (GPU off, file not found, bad setting, disk full), it raises `UpscalerError("what happened + how to fix")`.
3. The top-level functions are wrapped by `@friendly` (next concept), which catches **only** `UpscalerError` and prints it in a neat box.
4. Any *other* exception passes through untouched and shows a normal traceback. That's deliberate: it's a bug.
5. `NonFiniteOutput` is used as **control flow** inside the engine. The `Finisher` raises it when pixels are NaN, and the engine catches it and switches to full precision. The user never sees it.

What would go wrong without it

Either every error looks equally scary (beginners give up), or someone wraps everything in `except Exception: print("error")`, which hides real bugs and makes them impossible to debug.

Alternatives and trade-offs

- **Error codes / return values** (`return None, "error"`) are common in C and Go, but easy to forget to check.
- **One big try/except** around everything is simple but loses the difference between "user's fault" and "bug".
- A **hierarchy** of exceptions (`ConfigError`, `GPUError`, `DiskFullError`, all inheriting from `UpscalerError`) would allow finer handling. It wasn't needed here, but it's a good upgrade to mention.

Interview prep

> **Q: Why create your own exception class?**
>
> A: To separate expected problems the user can fix from real bugs. Expected ones raise UpscalerError with a plain-English fix, and the top-level wrapper prints those nicely. Anything else still shows a full traceback, so I don't hide bugs.
>
> **Q: Is using exceptions for control flow OK?**
>
> A: For rare, exceptional situations, yes. NonFiniteOutput only happens if half precision fails, so using an exception keeps the normal path simple. For things that happen every loop, I'd use normal return values instead.

Future use

In web backends, raise `NotFoundError` or `ValidationError` and have **error-handling middleware** turn them into HTTP 404 or 400 responses. That's the same pattern your `@friendly` decorator uses. Use it in CLIs, libraries and apps too.

9. Decorators (`@friendly`)

★★★ Interview favourite · Category: Python language / cross-cutting concerns

What it is

Analogy: gift wrapping. The present (your function) stays exactly the same, but the wrapping adds something around it: here, a safety net that catches expected errors.

Technically: a **decorator** is a function that takes a function and returns a new function that "wraps" it with extra behaviour. Writing `@friendly` above `def run_upscale` is shorthand for `run_upscale = friendly(run_upscale)`. It's used for **cross-cutting concerns**: behaviour many functions need (error handling, logging, timing, login checks) that you don't want to repeat in each one.

Where it is in your project

dev/engine.py · lines 47–59 · the decorator

```
47 def friendly(func):
48     """Run a top-level step; show UpscalerErrors as clean messages."""
49     def wrapper(*args, **kwargs):
50         try:
51             return func(*args, **kwargs)
52         except UpscalerError as e:
53             say("❌ STOPPED - here is what went wrong:", *str(e).splitlines())
54         except KeyboardInterrupt:
55             say("■  You stopped the cell.",
56                 "Any finished parts are saved. Run the cell again to continue",
57                 "from where it stopped.")
58     wrapper.__name__ = func.__name__
59     return wrapper
```

dev/engine.py · lines 1661–1663 · using it on run_upscale (also on check_setup, run_preview, clean_up)

```
1661 @friendly
1662 def run_upscale(source=None, allow_cpu=False):
1663     S = settings(source if source is not None else globals())
```

Why we used it

The four functions a user calls directly all need the same rule: "show UpscalerErrors as clean messages, and treat pressing Stop as a friendly pause". Writing that `try/except` four times would be repetitive and easy to get wrong.

How it works

1. `friendly(func)` receives the original function.
2. Inside, it defines `wrapper(*args, **kwargs)`. `*args, **kwargs` means "accept any arguments", which it passes straight on to `func`.
3. `try: return func(...)` runs the real function.
4. `except UpscalerError as e:` prints the message using `say(...)` instead of a traceback.
5. `except KeyboardInterrupt:` (the Stop button) reassures the user that progress is saved.
6. `wrapper.__name__ = func.__name__` keeps the original name, for nicer debugging. (The standard way is `@functools.wraps(func)`, which also copies the docstring. That's a small improvement you could make.)

7. `return wrapper`, and Python replaces the original function with the wrapped one.

What would go wrong without it

Duplicate error-handling code in four places, sooner or later one of them forgetting it, and beginners seeing scary tracebacks.

Alternatives and trade-offs

- A `try/except` inside each function: explicit, but repetitive.
- A **context manager** (`with friendly_errors():`): also fine, but the caller has to remember to use it.
- The decorator is the cleanest option here. Its downside is that it's "magic" for newcomers reading the code, and it currently **returns** `None` on error, which a caller could mistake for success. That's fine here, because nobody uses the return value.

Interview prep

> **Q: What is a decorator? Give an example from your project.**
>
> A: It's a function that wraps another function to add behaviour without changing it. My `@friendly` decorator wraps the four top-level steps. It catches my own UpscalerError and prints a clear message, and it turns the Stop button into a friendly "progress saved" note. Real bugs still raise normally.
>
> **Q: Where else are decorators used in real projects?**
>
> A: Flask and FastAPI use them for routes, like `@app.get("/users")`. They're also used for login checks (`@login_required`), caching (`@lru_cache`), retries, timing and logging.

Future use

Use one whenever several functions need the **same before/after behaviour**: authentication checks on API routes, timing, caching, retrying and logging.

10. Strategy pattern and duck typing (two interchangeable engines)

★★★ Interview favourite · Category: design patterns

What it is

Analogy: a power drill with swappable drill bits. The drill doesn't care which bit is attached, because every bit fits the same socket.

Technically: the **Strategy pattern** means several classes offer the same "socket" (the same methods), so the rest of the program can use any of them without knowing which one it has. In Python this often works by **duck typing** ("if it walks like a duck..."): no formal interface is declared. The classes simply have the same method names and attributes.

Where it is in your project

dev/engine.py · lines 893–893 · FastEngine.run

```
893     def run(self, before, core, after):
```

dev/engine.py · lines 1001–1001 · SteadyEngine.run — same signature

```
1001     def run(self, before, core, after):
```

dev/engine.py · lines 1135–1147 · build_engine picks the strategy

```
1135 def build_engine(S, device, info):
1136     out_w, out_h = output_size(info, S.SCALE)
1137     key = (S.MODEL, S.DENOISE_STRENGTH, S.USE_HALF_PRECISION, S.STEADY_CHUNK_FRAMES,
S.STEADY_WEIGHTS_FILE, str(device))
1138     engine = _ENGINE_CACHE.get(key)
1139     if engine is None:
1140         _ENGINE_CACHE.clear()
1141         free_memory()
1142         engine = (FastEngine if S.MODEL == "fast" else SteadyEngine)(S, device, out_w, out_h)
1143         _ENGINE_CACHE[key] = engine
1144     engine.finish = Finisher(out_w, out_h, S, device)   # new video size / blend / grain
1145     if S.MODEL == "steady":
1146         engine.calibrate(info)
1147     return engine
```

dev/engine.py · lines 1608–1615 · the caller works with either engine

```
1608                     while got < seg_frames:
1609                         k = min(engine.step, seg_frames - got)
1610                         core = reader.read_n(k)
1611                         if not core:
1612                             break
1613                         after = reader.peek_n(ctx) if ctx else []
1614                         for up in engine.run(before, core, after):
1615                             writer.write(face(up) if face else up)
```

Why we used it

You wanted two very different models: "fast" (frame by frame) and "steady" (needs neighbouring frames). The code that reads frames, saves progress and writes video shouldn't have to care which one is used. Otherwise every part of `process_video` and `run_preview` would be full of `if MODEL == "fast": … else: …`.

How it works

1. Both engines have the same "socket": `run(before, core, after)`, plus the attributes `step` (how many frames to read at a time), `context` (how many neighbouring frames they need) and `finish`.
2. `FastEngine.context = 0`, because it needs no neighbours. It simply ignores `before` and `after`.
3. `SteadyEngine.context = 5`, so the caller reads 5 extra frames ahead (`peek_n`) and keeps the last 5 as `before`.
4. `build_engine` chooses: `(FastEngine if S.MODEL == "fast" else SteadyEngine)(...)`, which picks a class and calls it in one line.
5. The main loop just calls `engine.run(before, core, after)`, whichever engine it is.

What would go wrong without it

`if/else` checks on the model would be spread across the code. Adding a third model would mean editing many places, and missing one would be a bug.

Alternatives and trade-offs

- An **abstract base class** (`class Engine(ABC)` with `@abstractmethod run`) or a `typing.Protocol` would make the "socket" explicit and checkable. That's a good improvement for bigger teams.
- **Inheritance** with shared code in a parent class: here the two engines share little code apart from the `Finisher`, so that was kept as a separate object instead (composition).
- Duck typing is quick and flexible, but nothing stops someone from writing a third engine that forgets `context`.

Interview prep

> **Q: Which design patterns did you use?**
>
> A: Mainly Strategy. Two upscaling engines share the same `run(before, core, after)` method and a `context` attribute that says how many neighbouring frames each one needs. A small factory function picks one from the settings, and the pipeline code works with either without any if-statements.
>
> **Q: How would you add a third model?**
>
> A: Write a new class with the same `run` method and `context` / `step` attributes, then add one line to `build_engine`. Nothing else changes. To make that safer, I'd define the interface as an abstract base class.

Future use

Use Strategy for **payment providers** (Stripe or PayPal), **storage backends** (local or S3), **export formats** (CSV, PDF or JSON), **sorting or pricing rules**, and swappable AI models.

11. PyTorch models and loading pretrained weights

★★★ Interview favourite (for AI/ML roles) · Category: machine learning

What it is

Analogy: a neural network is like a huge mixing desk with millions of knobs. The **architecture** is the desk's layout (which knobs exist and how they're wired). The **weights** are the knob positions that someone found by training for weeks. You can download the knob positions and set them on an identical desk.

Technically: in PyTorch, a model is a class that inherits from `nn.Module`. Its layers are created in `__init__`, and `forward()` describes how data flows through them. All learned numbers are kept in a `state_dict`, a dictionary from layer names to tensors (multi-dimensional arrays). `load_state_dict(…, strict=True)` copies downloaded weights in and **checks that every name and shape matches**.

Where it is in your project

dev/engine.py · lines 550–570 · SRVGGNetCompact (the "fast" model)

```
550 class SRVGGNetCompact(nn.Module):
551     """Real-ESRGAN 'realesr-general-x4v3' network (same layout as the official one)."""
552
553     def __init__(self, num_in_ch=3, num_out_ch=3, num_feat=64, num_conv=32, upscale=4):
554         super().__init__()
555         self.upscale = upscale
556         self.body = nn.ModuleList()
557         self.body.append(nn.Conv2d(num_in_ch, num_feat, 3, 1, 1))
558         self.body.append(nn.PReLU(num_parameters=num_feat))
559         for _ in range(num_conv):
560             self.body.append(nn.Conv2d(num_feat, num_feat, 3, 1, 1))
561             self.body.append(nn.PReLU(num_parameters=num_feat))
562         self.body.append(nn.Conv2d(num_feat, num_out_ch * upscale * upscale, 3, 1, 1))
563         self.upsampler = nn.PixelShuffle(upscale)
564
565     def forward(self, x):
566         out = x
567         for layer in self.body:
568             out = layer(out)
569         out = self.upsampler(out)
570         return out + F.interpolate(x, scale_factor=self.upscale, mode="nearest")
```

dev/engine.py · lines 849–858 · loading two weight files, mixing them, strict loading

```
849         a = torch_load(get_weights("general"))
850         b = torch_load(get_weights("general_wdn"))
851         a = a.get("params_ema", a.get("params", a))
852         b = b.get("params_ema", b.get("params", b))
853         d = float(S.DENOISE_STRENGTH)
854         # Mix the "strong denoise" and "keep noise" models (official Real-ESRGAN trick)
855         mixed = {k: d * a[k].float() + (1 - d) * b[k].float() for k in a}
856         self.net = SRVGGNetCompact()
857         self.net.load_state_dict(mixed, strict=True)
858         self.net.eval().to(device)
```

dev/engine.py · lines 927–941 · RealBasicVSR checkpoint: find the right prefix, check missing keys

```
        path = get_weights("realbasicvsr", S.STEADY_WEIGHTS_FILE)
        ck = torch_load(path)
        sd = ck.get("state_dict", ck) if isinstance(ck, dict) else ck
        for prefix in ("generator_ema.", "generator.", ""):
            sub = {k[len(prefix):]: v for k, v in sd.items() if k.startswith(prefix)}
            if any(k.startswith("basicvsr.") for k in sub):
                break
        self.net = RealBasicVSRNet()
        missing, unexpected = self.net.load_state_dict(sub, strict=False)
        missing = [k for k in missing if not k.endswith((".mean", ".std"))]
        if missing or unexpected:
            raise UpscalerError("The steady-model file does not match what this notebook expects.\n"
                                f"Missing: {missing[:5]}  Unexpected: {unexpected[:5]}\n"
                                'Please use MODEL = "fast" for now.')
        self.net.eval().to(device)
```

Why we used it

The official Real-ESRGAN Python package depends on an old library (`basicsr`) that **breaks** with new versions of PyTorch and torchvision. That was proven during development: it needed patches just to install. Instead, the network layouts were **re-written by hand (about 200 lines)**, copied from the official source code so that the layer names match exactly. Then the official weight files load directly. This means fewer dependencies and fewer surprises on Kaggle.

How it works

1. `SRVGGNetCompact.__init__` builds a list (`nn.ModuleList`) of 34 convolution layers with PReLU activations (a simple "bend" function) between them, ending in a `PixelShuffle` that turns channels into a 4× bigger picture.
2. `forward(x)`: pass the image through every layer, upsample, then **add a simple nearest-neighbour enlargement** of the input. So the network only has to learn the *difference* (a residual).
3. `torch_load` reads the `.pth` file. Weights are stored under `params` or `params_ema` (EMA means a smoothed average of weights from training, which usually gives better results).
4. `strict=True` means any mismatch in names or shapes raises an error immediately. That way you know the hand-written architecture is *exactly* right. (The test also compares the output to a plain enlargement to check it's sensible.)
5. For RealBasicVSR, the checkpoint comes from a training framework (OpenMMLab), so keys look like `generator_ema.basicvsr.…`. The code strips the prefix, loads with `strict=False`, then **checks the missing list itself** (ignoring two harmless constants), and raises a clear error otherwise.
6. `.eval()` switches the model to inference mode, and `.to(device)` moves it to the GPU.

What would go wrong without it

With the official package: installation failures on Kaggle as versions change. With non-strict loading and no check: a wrongly built network would *load silently* and produce garbage pictures, and you might not notice until a long run finishes.

Alternatives and trade-offs

- **Use the official package** (`pip install realesrgan`). It's less code to own, but has dependency breakage (and it really broke during development).
- **Hugging Face / spandrel**-style model loaders that support many architectures. That's convenient, but it's another dependency.

- **Export to ONNX / TensorRT** for speed. That's more setup.
- *Trade-off:* your re-written RealBasicVSR couldn't be tested with the real weight file during development (the download was blocked there). The code compensates with strict key checks and a clear fallback message.

Interview prep

> **Q: What is a state_dict?**
> A: It's a dictionary of all the learned numbers in a PyTorch model, keyed by layer name, like "body.0.weight". To load pretrained weights, you build the same architecture and call load_state_dict. I use strict loading so any mismatch fails immediately.
>
> **Q: Why did you re-implement the architectures?**
> A: The official packages depend on an unmaintained library that breaks with current PyTorch. The networks are small, so I wrote them in plain PyTorch with identical layer names. The official weights load strictly, and I removed a whole chain of fragile dependencies.
>
> **Q: What's the difference between model.eval() and torch.no_grad()?**
> A: eval() changes how some layers behave, like dropout and batch norm, to inference mode. no_grad() stops PyTorch from recording operations for training, which saves memory and time. For inference you want both.

Future use

Any time you **use a pretrained model**: image classifiers, speech recognition, text models. Also any time a dependency is fragile and the part you need is small, owning a small copy can be better than depending on a big package.

12. Automated end-to-end testing with fake data and test doubles

★★★ Interview favourite · Category: testing

What it is

Analogy: a fire drill. You don't wait for a real fire to find out whether the alarms and exits work. You run a realistic practice with a fake fire.

Technically: an **automated test** is code that runs your code and checks the results with **assertions** (`assert condition`: stop with an error if it isn't true). An **end-to-end (E2E) test** runs the whole flow (input video → output video), not just one small function. **Synthetic test data** is fake input generated on the spot. A **test double** is a fake replacement for something expensive or unavailable.

Where it is in your project

dev/test_local.py · lines 68–76 · test: restarting mid-video gives exactly the right frames

```
 68     # 2) frame-exact restart in the middle (what 'continue' relies on)
 69     info = E.probe_video(clip)
 70     allf = decode_all(clip)
 71     assert len(allf) == info["est_frames"], (len(allf), info["est_frames"])
 72     for k in (1, 37, 64, 110):
 73         r = subprocess.run(E.decode_command(info, start_frame=k), capture_output=True, check=True)
 74         part = np.frombuffer(r.stdout, np.uint8).reshape(-1, info["height"], info["width"], 3)
 75         assert len(part) == len(allf) - k and np.array_equal(part[0], allf[k]) and
np.array_equal(part[-1], allf[-1]), k
 76     print("OK frame-exact restart")
```

dev/test_local.py · lines 108–116 · test: stop partway, then continue

```
108     # 5) full run, interrupted by the time limit, then continued; compare with an uninterrupted run
109     Sx = E.settings(base_settings(SAVE_EVERY_SECONDS=1))
110     st1 = E.process_video(clip, Sx, dev, deadline=time.time() + 1.5)
111     work_root = os.path.join(OUT, E.WORK_FOLDER_NAME)
112     prog = json.load(open(next(os.path.join(r, f) for r, _, fs in os.walk(work_root) for f in fs if f
== "progress.json")))
113     print("first run:", st1, prog["frames_done"], "frames,", len(prog["segments"]), "parts")
114     assert st1 == "paused" and 0 < prog["frames_done"] < len(allf)
115     st2 = E.process_video(clip, Sx, dev, deadline=time.time() + 3600)
116     assert st2 == "done"
```

dev/test_local.py · lines 138–146 · test double: a fake RealBasicVSR checkpoint with random weights

```
138     torch.manual_seed(0)
139     net = E.RealBasicVSRNet()
140     for p in net.parameters():
141         p.data.mul_(0.1)
142     ck = {"meta": {}, "state_dict": {**{"generator." + k: v for k, v in net.state_dict().items()},
143                                      **{"generator_ema." + k: v for k, v in
net.state_dict().items()},
144                                      "step_counter": torch.zeros(1)}}
145     wpath = os.path.join(TMP, "fake_realbasicvsr.pth")
146     torch.save(ck, wpath)
```

Why we used it

There was **no GPU** during development, and the RealBasicVSR weights couldn't be downloaded. The test runs everything on a normal CPU using **tiny 160×96 videos** generated by ffmpeg's built-in test pattern (`testsrc2`). That made it possible to prove the hardest parts before you ever opened Kaggle: frame-exact resume, audio, frame rate, colours, rotation, HDR, tiling seams and error messages.

How it works

1. `ff(...)` generates test videos with ffmpeg (`-f lavfi -i testsrc2=...`), including a **rotated** one and an **HDR** one, to test the phone-video edge cases.
2. **Resume test:** decode all frames, then decode starting at frames 1, 37, 64 and 110, and `assert` the first frame equals the right one (`np.array_equal`).
3. **Tiling test:** upscale with and without tiles and assert the difference is at most 2 out of 255 (no visible seams).
4. **Pause/continue test:** call `process_video` with a deadline only 1.5 s away (forcing a pause), resume it, then compare against an uninterrupted run, **frame by frame, exactly equal**.
5. **Test double:** build a `RealBasicVSRNet` with random weights, save it in the **same format** as the official checkpoint (`generator_ema.` prefixes), and run the steady pipeline. This tests the plumbing, not the picture quality.
6. **Error tests:** wrong folder, wrong MODEL and missing file must print friendly messages.
7. `allow_cpu=True` and the environment variables `TEST_DIR` and `MODEL_DIR` let the same code run outside Kaggle. This is a small example of making code **testable**.

What would go wrong without it

Bugs would only be found on Kaggle, after waiting for a GPU session, possibly hours into a run. Changing one thing later (say, colour handling) could silently break resume, and you'd never know.

Alternatives and trade-offs

- **pytest** is the standard Python test runner. You'd split the script into many small `test_…` functions with clearer reports. This script is one long function, so the first failure stops everything else.
- **Unit tests** for small functions (like `settings_label` or `output_size`) are fast and precise, but miss integration bugs. A real project uses both.
- **CI (continuous integration)**, such as GitHub Actions, runs the tests automatically on every push. Your project doesn't have this yet.
- *Honest limits:* the tests don't check speed or real image quality on a GPU.

Interview prep

> **Q: How did you test something that needs a GPU, without a GPU?**
>
> A: I made the code device-agnostic, so it runs on the CPU when allowed, and generated tiny synthetic videos with ffmpeg. For the model whose weights I couldn't download, I used a test double: random weights saved in the official checkpoint format. That tests all the plumbing.
>
> **Q: What's your most valuable test?**
>
> A: The pause-and-continue test. It forces a stop partway through, resumes, and checks the result is identical to an uninterrupted run. That proves the riskiest feature, resuming after Kaggle's time limit, works frame-exactly.
>
> **Q: What would you improve in your tests?**
>
> A: Move to pytest with separate test functions, add fast unit tests, and run them in GitHub Actions on every push.

Future use

Always write at least one **end-to-end "happy path" test** plus tests for the **riskiest feature**. Use **fake data** and **test doubles** (mocks, fakes, stubs) for slow, expensive or unavailable things: payment APIs, email, GPUs, external databases.

Part 2B — Important concepts (★★)

Solid engineering ideas you should be able to explain. They are grouped loosely: reliability first, then design, performance, machine learning, video, and tooling.

13. Cleanup on failure: try / except / finally and context managers

★★ Important · Category: Python / resource management

What it is

Analogy: when you leave a rented flat, you return the keys whether the stay went well or badly. `finally` is "always return the keys".

Technically: `try:` runs code, `except:` handles errors, and `finally:` runs **no matter what** (success, error, or Stop pressed). A **context manager** (`with open(...) as f:`) is a tidy way to do the same for one resource: it closes the file automatically. Your code also uses the **catch, clean up, re-raise** pattern: `except BaseException: cleanup(); raise`.

Where it is in your project

dev/engine.py · lines 1607–1623 · if anything goes wrong while writing a part, delete that half-written part, then re-raise

```
1607                try:
1608                    while got < seg_frames:
1609                        k = min(engine.step, seg_frames - got)
1610                        core = reader.read_n(k)
1611                        if not core:
1612                            break
1613                        after = reader.peek_n(ctx) if ctx else []
1614                        for up in engine.run(before, core, after):
1615                            writer.write(face(up) if face else up)
1616                        bar.update(len(core))
1617                        before = (before + core)[-ctx:] if ctx else []
1618                        got += len(core)
1619                        if len(core) < k:
1620                            break
1621                except BaseException:
1622                    writer.abort()
1623                    raise
```

dev/engine.py · lines 1638–1640 · always close the progress bar and stop the ffmpeg decoder

```
1638        finally:
1639            bar.close()
1640            reader.close()
```

dev/engine.py · lines 1044–1050 · always change back to the original folder

```
1044        here = os.getcwd()
1045        os.chdir(MODEL_DIR)   # its helper files download into the current folder - keep them out of Output
1046        try:
1047            self.restorer = GFPGANer(model_path=weights, upscale=1, arch="clean",
1048                                     channel_multiplier=2, bg_upsampler=None, device=device)
1049        finally:
1050            os.chdir(here)
```

Why we used it

The pipeline holds **real resources**: running ffmpeg processes, threads, open log files, half-written video files and the current working folder. If an error or the Stop button skipped cleanup, you'd get *zombie* ffmpeg processes (still running, still using CPU), locked files, and broken parts that could later be mistaken for good ones.

How it works

1. `except BaseException:` catches *everything*, including `KeyboardInterrupt` (the Stop button), which is **not** a subclass of the normal `Exception`.
2. `writer.abort()` kills the encoder and deletes the half-written part.
3. `raise` with no argument re-throws the **same** error, so it's not swallowed: `@friendly` or the user still sees it.
4. `finally:` around the whole loop always closes the progress bar and the reader (which kills ffmpeg and drains its queue).
5. `with open(tmp, "w") as f:` closes files even if `json.dump` fails.
6. `with torch.no_grad(), torch.autocast(...)` turns on "no training bookkeeping" and "half precision" only for that block, then switches them off.

What would go wrong without it

Orphaned background processes eating CPU, files left open, the Stop button leaving a corrupt segment that resume trusts, or GFPGAN leaving your notebook running in the wrong folder.

Alternatives and trade-offs

`contextlib.ExitStack`, or making `FrameReader`/`VideoWriter` into context managers themselves (adding `__enter__`/`__exit__`), would make the cleanup even harder to forget. That's a nice upgrade. Catching `BaseException` is normally discouraged, and is only right here because it **re-raises** immediately after cleaning up.

Interview prep

> **Q: Why catch BaseException? Isn't that bad practice?**
> A: Swallowing it would be. I only use it to clean up (delete the half-written file, kill ffmpeg) and then re-raise immediately. Catching Exception alone would miss KeyboardInterrupt, the Stop button, which is exactly when cleanup matters most.
>
> **Q: What's the difference between except and finally?**
> A: except runs only when there's an error. finally runs always, so it's for cleanup like closing files and processes.

Future use

Use it for database connections and transactions (roll back on error), file handles, network sockets, locks, temporary files, and anything you "open" that must be "closed".

14. Idempotency: safe to run again

★★ Important · Category: reliability

What it is

Analogy: a lift button. Pressing it five times doesn't call five lifts. The result is the same as pressing it once.

Technically: an operation is **idempotent** if running it again doesn't change the result or repeat work. For beginners pressing "Run All" repeatedly, this matters a lot.

Where it is in your project

dev/engine.py · lines 1539–1543 · finished video already there → skip

```
1539    final_name = f"{info['stem']}_upscaled_{settings_label(S)}.mp4"
1540    final_path = os.path.join(out_dir, final_name)
1541    if os.path.exists(final_path):
1542        print(f"✅ '{final_name}' already exists - skipping. (Delete it to redo.)")
1543        return "done"
```

dev/engine.py · lines 1144–1147 · engine reused, only the per-video parts refreshed

```
1144    engine.finish = Finisher(out_w, out_h, S, device)   # new video size / blend / grain
1145    if S.MODEL == "steady":
1146        engine.calibrate(info)
1147    return engine
```

Why we used it

On Kaggle you often re-run cells: after a crash, after changing one setting, or in a new session. Without idempotency, re-running could re-do hours of work, or create duplicate or conflicting files.

How it works

1. The output file name contains the settings (`settings_label`), for example `holiday_upscaled_2x_fast_dn0.5_blend0.1.mp4`. Same settings mean the same name.
2. If that file exists, the video is skipped. That's safe because the file only appears when it's complete (atomic rename, concept 4).
3. Unfinished work is continued instead of restarted (checkpointing, concept 3).
4. Different settings get a different work folder (via a hash, concept 16), so they never mix.
5. Previews have the settings and start time in their names, so different tests don't overwrite each other.

What would go wrong without it

Re-running would waste GPU hours, overwrite good results, or mix parts made with different settings into one broken video.

Alternatives and trade-offs

Asking "overwrite? y/n" doesn't work well in notebooks. A `FORCE_REDO` setting would be a nice addition. *Trade-off:* if you *want* to redo a video with identical settings, you must delete the old file first, and the message tells you so.

Interview prep

> **Q: What does idempotent mean, and where did you need it?**
>
> A: Running something twice gives the same result as once. Users re-run notebook cells all the time, so finished videos are skipped, unfinished ones resume, and different settings get separate names and folders, so nothing is repeated or mixed.

Future use

Payment APIs (charging twice is a disaster: use idempotency keys), **database migrations**, **setup scripts**, **message processing** (a message may arrive twice), and HTTP `PUT` / `DELETE`.

15. Deadlines and graceful shutdown (the 12-hour budget)

★★ Important · Category: reliability / scheduling

What it is

Analogy: a shop closes at 6 pm, so you stop taking new customers at 5:45 and finish serving the ones inside, instead of being kicked out mid-sale.

Technically: the program knows its **time budget** and checks before each new piece of work whether there's enough time left to finish it. If not, it stops **cleanly**, with its state saved, before being force-killed.

Where it is in your project

dev/engine.py · lines 1666–1666 · deadline = session start + 11.5 h − 10 min safety margin

```
1666    deadline = session_start_time() + float(S.MAX_RUNTIME_HOURS) * 3600 - 10 * 60
```

dev/engine.py · lines 1593–1597 · before each part: is there time for one more?

```
1593            while True:
1594                avg = sum(seg_times[-5:]) / len(seg_times[-5:]) if seg_times else 0
1595                if time.time() + avg * 1.3 > deadline:
1596                    status = "paused"
1597                    break
```

dev/engine.py · lines 1226–1231 · when did this session start?

```
1226 def session_start_time():
1227     try:
1228         import psutil
1229         return min(psutil.Process(1).create_time(), psutil.Process().create_time())
1230     except Exception:
1231         return _ENGINE_LOADED_AT
```

Why we used it

Kaggle kills a session at 12 hours. In a background "Save Version" run, a session killed *mid-write* may lose its output. Stopping just before the limit means the run ends normally, Kaggle saves the Output (with the progress), and the next run can continue.

How it works

1. `session_start_time()` asks `psutil` when the container's first process (PID 1) and the notebook kernel started, and takes the earlier one. If `psutil` is missing, it falls back to the time the ENGINE cell ran.
2. The deadline is 11.5 h (`MAX_RUNTIME_HOURS`) after that, minus 10 minutes to leave time for joining parts.
3. It keeps the time each segment took (`seg_times`) and averages the last 5.
4. Before starting a new segment: if `now + average × 1.3 > deadline`, it stops with status `"paused"`. The ×1.3 is a safety margin for slow segments.
5. `run_upscale` then prints how to continue.

What would go wrong without it

The session would be killed at a random point. In a "Save Version" run that can mean no saved output at all, so hours of work and progress would vanish.

Alternatives and trade-offs

Listening for a termination signal (`SIGTERM`) is the usual approach on servers (Kubernetes gives you a warning), but Kaggle doesn't promise one. *Weakness:* the start-time detection is a **heuristic** (an educated guess). If Kaggle changes how containers start, the budget could be off. `MAX_RUNTIME_HOURS` lets the user adjust it.

Interview prep

> **Q: How do you deal with a hard time limit you don't control?**
> A: I track how long each unit of work takes, and before starting the next one, I check that it can finish before a safe deadline. If not, I stop cleanly with progress saved, and the next session resumes from there.

Future use

Use it for **serverless functions** (AWS Lambda has a 15-minute limit), **CI jobs with timeouts**, **cron jobs** that must finish before the next run, mobile background tasks, and API requests with time limits.

16. Hashing settings into a cache key

★★ Important · Category: caching / data integrity

What it is

Analogy: a fingerprint. A short code that is (almost certainly) unique to one exact combination of settings.

Technically: a **hash function** (here **SHA-1**) turns any data into a fixed-length code. The same input always gives the same code, and a tiny change gives a completely different one. Using it to name the work folder means each settings combination gets its own folder.

Where it is in your project

dev/engine.py · lines 151–159 · settings_key

```
151 def settings_key(S):
152     """A short code that changes whenever a setting that affects the picture changes."""
153     parts = dict(scale=S.SCALE, model=S.MODEL, blend=S.ORIGINAL_BLEND, grain=S.FILM_GRAIN,
154                  face=S.FACE_RESTORE, crf=S.VIDEO_QUALITY, preset=S.ENCODER_SPEED, v=1)
155     if S.MODEL == "fast":
156         parts["denoise"] = S.DENOISE_STRENGTH
157     if S.FACE_RESTORE:
158         parts["face_strength"] = S.FACE_STRENGTH
159     return hashlib.sha1(json.dumps(parts, sort_keys=True).encode()).hexdigest()[:8]
```

dev/engine.py · lines 1544–1545 · the work folder is named video + hash

```
1544     work_root = os.path.join(out_dir, WORK_FOLDER_NAME)
1545     work = os.path.join(work_root, f"{info['stem']}__{settings_key(S)}")
```

Why we used it

If you change, say, the denoise strength halfway through a video and run again, the old parts (made with the old setting) must **not** be glued together with new ones. That would give a video whose look suddenly changes. The hash makes "different settings → different progress folder" automatic.

How it works

1. It collects **only the settings that change the picture** (scale, model, blend, grain, quality…). Settings like the preview start time don't matter here.
2. `json.dumps(parts, sort_keys=True)` turns the dictionary into text **in a fixed order**. Without `sort_keys`, the same settings could produce different text and a different hash.
3. `hashlib.sha1(...).hexdigest()[:8]` keeps the first 8 hex characters, which is plenty to avoid clashes between a few settings combinations.
4. `v=1` is a **version number** inside the key. If the processing logic ever changes, bumping it to 2 automatically starts fresh folders.

What would go wrong without it

Mixed-settings videos, or someone has to manually delete progress after every settings change (and will forget).

Alternatives and trade-offs

You could put all settings in the folder name (`settings_label`). That's readable, but long and fragile with many settings. SHA-1 is no longer safe for **security** (signatures), but it's perfectly fine for naming. `hashlib.sha256` would be the habit-forming choice.

Interview prep

> **Q: Why hash the settings?**
> A: To key the saved progress by the exact settings that affect the output. I serialise them as sorted JSON so the hash is stable, and include a version number. If settings change, a new folder is used automatically, so old parts are never mixed in.

Future use

Use it for **cache keys** (cache an API result by hashing the request), build systems, **file deduplication** (hash file contents), ETags in HTTP, and detecting whether a config changed.

17. Factory function + memoization (build the engine once)

★★ Important · Category: design patterns / performance

What it is

Analogy: a coffee machine that heats up once in the morning. After that it makes every cup quickly, instead of being switched off and on for each cup.

Technically: a **factory** is a function whose job is to create the right object (here, the right engine). **Memoization / caching** means remembering a result so the next identical request reuses it instead of recomputing it.

Where it is in your project

dev/engine.py · lines 1132–1147 · build_engine: create only if the key changed, else reuse

```
1132 _ENGINE_CACHE = {}
1133
1134
1135 def build_engine(S, device, info):
1136     out_w, out_h = output_size(info, S.SCALE)
1137     key = (S.MODEL, S.DENOISE_STRENGTH, S.USE_HALF_PRECISION, S.STEADY_CHUNK_FRAMES,
S.STEADY_WEIGHTS_FILE, str(device))
1138     engine = _ENGINE_CACHE.get(key)
1139     if engine is None:
1140         _ENGINE_CACHE.clear()
1141         free_memory()
1142         engine = (FastEngine if S.MODEL == "fast" else SteadyEngine)(S, device, out_w, out_h)
1143         _ENGINE_CACHE[key] = engine
1144     engine.finish = Finisher(out_w, out_h, S, device)   # new video size / blend / grain
1145     if S.MODEL == "steady":
1146         engine.calibrate(info)
1147     return engine
```

dev/engine.py · lines 180–194 · ffmpeg_info: ask ffmpeg about itself only once

```
180 _FFMPEG_INFO = {}
181
182
183 def ffmpeg_info():
184     if _FFMPEG_INFO:
185         return _FFMPEG_INFO
186     if not shutil.which("ffmpeg") or not shutil.which("ffprobe"):
187         raise UpscalerError("ffmpeg (the video tool) is not installed. Run the CHECK cell first - "
188                             "it installs it automatically if needed.")
189     out = subprocess.run(["ffmpeg", "-hide_banner", "-version"], capture_output=True,
text=True).stdout
190     m = re.search(r"version\s+n?(\d+)", out)
191     major = int(m.group(1)) if m else 4
192     filters = subprocess.run(["ffmpeg", "-hide_banner", "-filters"], capture_output=True,
text=True).stdout
193     _FFMPEG_INFO.update(major=major, zscale=" zscale " in filters, tonemap=" tonemap " in filters)
194     return _FFMPEG_INFO
```

Why we used it

Loading a model means reading weight files, building the network and copying it to the GPU, which takes seconds. You often run PREVIEW several times and then UPSCALE on several videos. Reloading every time would waste time and **GPU memory** (two copies could even cause an out-of-memory error).

How it works

1. The cache key is the tuple of settings that change the *model itself* (model type, denoise, precision, chunk size, weights file, device).
2. On a **cache hit**, the same engine is reused.
3. On a **miss**, `_ENGINE_CACHE.clear()` and `free_memory()` throw away the old model **first**, so two models never fill the GPU at once.
4. Things that change **per video** (output size, blend, grain) are refreshed every time: `engine.finish = Finisher(...)`.
5. `ffmpeg_info()` stores its answer in a module-level dictionary, so later calls return instantly.

What would go wrong without it

Slow repeated loading, doubled GPU memory use, and possible out-of-memory crashes when you switch between preview and full runs.

Alternatives and trade-offs

`functools.lru_cache` does memoization with one decorator, but it can't easily say "clear the old one first to free the GPU". **Global mutable state** (module-level dictionaries) is simple, but it makes code harder to test and isn't thread-safe. An `Upscaler` class holding its own cache would be cleaner in a bigger project.

Interview prep

> **Q: Where do you use caching?**
> A: Loading the AI model is slow and uses GPU memory, so a factory function caches the engine keyed by the settings that affect the model. Per-video options are refreshed each time. When the key changes, I free the old model before loading the new one, so the GPU never holds two.

Future use

Use it for **database connection pools**, loading ML models in a web API (load once at startup), expensive config parsing, API responses (with an expiry time), and computed values.

18. Lazy loading and lazy installation

★★ Important · Category: performance / dependencies

What it is

Analogy: you only pack an umbrella if the forecast says rain.

Technically: lazy means "don't do work until it's actually needed". Your project only installs and loads the face-restoration add-on (GFPGAN and its helper libraries, about 300 MB) **if** `FACE_RESTORE = True`. It also only downloads the model files for the model you chose.

Where it is in your project

dev/engine.py · lines 1153–1159 · build_face: nothing happens if faces are off

```
1153 def build_face(S, device):
1154     if not S.FACE_RESTORE:
1155         return None
1156     if "face" not in _FACE_CACHE:
1157         _FACE_CACHE["face"] = FaceRestorer(S, device)
1158     _FACE_CACHE["face"].strength = float(S.FACE_STRENGTH)
1159     return _FACE_CACHE["face"]
```

dev/engine.py · lines 1072–1077 · install only if not already importable

```
1072 def ensure_gfpgan_installed():
1073     try:
1074         import gfpgan  # noqa: F401
1075         return
1076     except Exception:
1077         pass
```

dev/engine.py · lines 1267–1269 · the CHECK cell downloads only what your settings need

```
1267     needed = ["general", "general_wdn"] if S.MODEL == "fast" else ["realbasicvsr"]
1268     if S.FACE_RESTORE:
1269         needed.append("gfpgan")
```

Why we used it

Most people will never turn on face restoration (it's off by default). Installing it for everyone would add minutes of setup and another place for things to break, for nothing.

How it works

1. `build_face` returns `None` when faces are off, and the main loop does `face(up) if face else up`.
2. When it is on, `ensure_gfpgan_installed()` first tries `import gfpgan`. If that works, it stops. Otherwise it installs.
3. The import inside the function (`from gfpgan import GFPGANer`) is a **lazy import**: Python only loads the library when that line runs.
4. The `_FACE_CACHE` keeps it loaded for the rest of the session.

What would go wrong without it

Slower start for everyone, more failure points, more disk use, and an install error that blocks users who never wanted the feature.

Alternatives and trade-offs

Install everything up front: simpler and more predictable, but slower. *Trade-off:* lazy steps fail later. If GFPGAN can't install, you only find out when you turn it on. That's why the CHECK cell runs the downloads for the features you enabled *before* the long run.

Interview prep

> **Q: What is lazy loading?**
> A: Doing expensive work only when it's needed. My optional face-restoration feature installs and loads its 300 MB of dependencies only when the user enables it, and model files are only downloaded for the chosen model.

Future use

Use it for **lazy-loading images** on websites (load as you scroll), loading app screens or modules on demand (code splitting), optional plugins, and database relations loaded on first access.

19. Facade functions and separation of concerns

★★ Important · Category: architecture

What it is

Analogy: a restaurant menu. You order "pizza" and don't need to know the kitchen's steps. The kitchen is also split into stations (dough, oven, toppings), and each does one job.

Technically: a **facade** is a simple front door to a complicated system: `run_preview()`, `run_upscale()`, `check_setup()`, `clean_up()`. **Separation of concerns** means each part of the code has *one* job and knows little about the others.

Where it is in your project

dev/engine.py · lines 1662–1672 · run_upscale: a short, readable top-level story

```
def run_upscale(source=None, allow_cpu=False):
    S = settings(source if source is not None else globals())
    device = get_device(allow_cpu)
    videos = find_videos(S)
    deadline = session_start_time() + float(S.MAX_RUNTIME_HOURS) * 3600 - 10 * 60
    left = deadline - time.time()
    print(f"{len(videos)} video(s) to do. Time left in this session (safe limit):
{human_time(left)}")
    results = {}
    for v in videos:
        status = process_video(v, S, device, deadline)
        results[os.path.basename(v)] = status
```

Part	Single job
`settings()`	read and validate user choices
`probe_video()`	facts about the input video
`FrameReader`	video → frames
`FastEngine` / `SteadyEngine`	frames → 4× bigger frames
`Finisher`	resize, blend, grain, then convert to bytes
`VideoWriter`	frames → MP4 part
`save_progress()` / `assemble()`	checkpoint / final video

Why we used it

The notebook cells are one line each (`run_upscale()`), which beginners can't break. The separation made features cheap to add: the preview reuses the reader, engine, finisher and writer, and only adds the side-by-side part.

How it works

Each component only receives what it needs. For example `VideoWriter` gets a path, a size, a frame rate and settings, and knows nothing about AI models. The engine knows nothing about files. The top-level functions connect the pieces in order.

What would go wrong without it

One huge function where changing the encoder could break the model code, no reuse between preview and full runs, and very hard testing.

Alternatives and trade-offs

Honest weakness: everything lives in **one 1,700-line file**. That's a deliberate choice (the notebook needs one ENGINE cell), but in a normal project you would split it into modules (`io.py`, `models.py`, `pipeline.py`, `cli.py`). `build_notebook.py` could then combine them.

Interview prep

> **Q: How is your code organised?**
> A: As a pipeline of small components, each with one job: read frames, upscale, finish, write, checkpoint. Simple facade functions like run_upscale tie them together for the user. The preview reuses the same components. I kept it in one file because the notebook needs one code cell, but I'd split it into modules in a larger project.

Future use

This is the same idea as **controllers → services → repositories** in web backends, and **MVC/MVVM** in apps. Keep "what the user calls" thin, and keep each job in its own class or module.

20. Configuration with defaults (and notebook globals)

★★ Important · Category: configuration

What it is

Analogy: a new phone comes with sensible factory settings, and you only change the ones you care about.

Technically: all options have **defaults** in one dictionary (`DEFAULTS`). The user's values override them. Here the user's values come from the **global variables** of the notebook's SETTINGS cell, because all cells share one Python "namespace" (one big shared set of variables).

Where it is in your project

dev/engine.py · lines 81–90 · one place for every option and its default

```
81 DEFAULTS = dict(
82     INPUT_FOLDER="/kaggle/input", ONLY_THESE_FILES=[], OUTPUT_FOLDER="/kaggle/working",
83     SCALE=2, MODEL="fast", DENOISE_STRENGTH=0.5,
84     ORIGINAL_BLEND=0.10, FILM_GRAIN=0.0,
85     FACE_RESTORE=False, FACE_STRENGTH=0.5,
86     PREVIEW_VIDEO="", PREVIEW_START_SECONDS=0, PREVIEW_LENGTH_SECONDS=8,
87     VIDEO_QUALITY=17, ENCODER_SPEED="medium",
88     STEADY_CHUNK_FRAMES=30, SAVE_EVERY_SECONDS=10, MAX_RUNTIME_HOURS=11.5,
89     USE_HALF_PRECISION=True, STEADY_WEIGHTS_FILE="", SKIP_PREVIEW_IN_SAVE_VERSION=True,
90 )
```

dev/engine.py · lines 99–104 · read the notebook's variables, falling back to defaults

```
 99 def settings(source=None):
100     """Read the settings from the SETTINGS cell and check they make sense."""
101     g = source if source is not None else globals()
102     S = Settings()
103     for k, v in DEFAULTS.items():
104         setattr(S, k, g.get(k, v))
```

Why we used it

Beginners can delete or forget a line in SETTINGS and the code still works. Tests can pass a plain dictionary instead (`settings(source=my_dict)`), which is how `test_local.py` runs different configurations.

How it works

1. `g = source if source is not None else globals()`: by default it uses the notebook's variables.
2. For every known option, `g.get(name, default)` takes your value or the default.
3. The values go onto a `Settings` object, so other code can write `S.SCALE` instead of `S["SCALE"]`.
4. Then validation runs (concept 7).

What would go wrong without it

A missing variable would raise `NameError` deep in the code, and tests would need to fake a whole notebook.

Alternatives and trade-offs

- **Environment variables / `.env` files** are standard for servers and secrets. Your project has no secrets, so it doesn't need them.

- **YAML/JSON config files**, `argparse` (command-line flags), or `pydantic-settings`.
- *Weakness:* reading `globals()` is **implicit**. It's not obvious from a function call where its inputs come from, and a typo in a variable name (`SCAEL = 3`) is silently ignored. A `@dataclass Settings` with type hints would be cleaner.

Interview prep

> **Q: How is the app configured?**
> A: There's one table of defaults, and the user overrides values in a settings cell. The engine reads those variables with defaults, validates them, and puts them on a settings object. Tests pass a dictionary instead, so the same code runs in tests and on Kaggle.

Future use

Every app needs configuration: database URLs, API keys (through environment variables, never in code), feature flags, and limits. Always have **defaults**, **one place** for config, and **validation** at startup.

21. Sliding window with overlap (context frames)

★★ Important · Category: algorithms

What it is

Analogy: reading a long story through a small window that shows a few lines at a time. To understand the middle lines properly, you also glance at the lines just above and below.

Technically: the "steady" model works better when it sees neighbouring frames. The whole video can't fit on the GPU, so it is processed in **windows**: a *core* of frames to output, plus some *before* and *after* frames as **context** that are looked at but not output. Windows overlap so there are no jumps at the joins.

Where it is in your project

dev/engine.py · lines 1001–1012 · split into chunks, each with up to 5 context frames on each side

```
1001     def run(self, before, core, after):
1002         ctx = self.context
1003         before, after = list(before[-ctx:]) if ctx else [], list(after[:ctx])
1004         if len(core) > self.chunk:
1005             out = []
1006             for s in range(0, len(core), self.chunk):
1007                 b = (list(before) + list(core[:s]))[-ctx:]
1008                 a = (list(core[s + self.chunk:]) + after)[:ctx]
1009                 out += self.run(b, core[s:s + self.chunk], a)
1010             return out
1011         try:
1012             return self._window(before + list(core) + after, len(before), len(core))
```

dev/engine.py · lines 381–384 · peek ahead: read frames and push them back for next time

```
381     def peek_n(self, n):
382         frames = self.read_n(n)
383         self.pending.extendleft(reversed(frames))
384         return frames
```

dev/engine.py · lines 1613–1617 · the caller keeps the last 5 frames as the next "before"

```
1613                         after = reader.peek_n(ctx) if ctx else []
1614                         for up in engine.run(before, core, after):
1615                             writer.write(face(up) if face else up)
1616                         bar.update(len(core))
1617                         before = (before + core)[-ctx:] if ctx else []
```



The model looks at all 18 frames, but only outputs the 8 core frames. Next chunk: core frames 8–12 become its "before".

Figure 6 — Sliding window: before / core / after frames (steady model).

Why we used it

Without context, the first and last frames of every chunk would only "see" one direction in time. Their quality would differ from the middle frames, and you'd get a visible "pulse" every 30 frames, which is exactly the flicker the steady model is meant to remove.

How it works

1. The main loop reads `core` frames, then **peeks** at the next 5 (`after`) without removing them. They'll be read again as the next core.
2. It passes the last 5 frames of the previous core as `before`.
3. `SteadyEngine.run` builds `before + core + after`, runs the model on all of it, and keeps only the core outputs (`keep_from`, `keep_count`).
4. On resume, the decoder starts 5 frames early (`rstart = start - ctx`) so the first window has context too.
5. The fast model has `context = 0`, so the same loop just skips all of this.

What would go wrong without it

Visible jumps or flicker at chunk boundaries, and worse quality at the start and end of every chunk.

Alternatives and trade-offs

Bigger overlap gives better quality but more wasted work (5 + 5 extra frames per 30 is about 33% extra for the first stage). Processing the whole video at once is impossible on 16 GB of memory. *Honest note:* the official model can carry information across the *entire* video, while windows limit that to about 5 frames of extra context. That's a small quality cost in exchange for fitting in memory.

Interview prep

> **Q: How do you process a video that doesn't fit in GPU memory, with a model that needs neighbouring frames?**
>
> A: With overlapping windows. Each chunk includes a few context frames before and after, and I only keep the outputs of the core frames. Peeking ahead and carrying the last few frames forward keeps the joins seamless, even across resume.

Future use

Use it for **time-series** data (moving averages, anomaly detection), **text chunking for AI** (overlapping chunks so sentences aren't cut), audio processing, and streaming analytics.

22. Batching

★★ Important · Category: performance

What it is

Analogy: a washing machine. Washing 4 shirts at once takes about as long as washing 1.

Technically: a GPU is fastest when given several items at once (a **batch**), because it does thousands of small calculations in parallel. Your "fast" engine sends **4 frames at a time**. The steady engine uses chunks of 30.

Where it is in your project

dev/engine.py · lines 859–861 · batch of 4 on GPU, 1 on CPU; read 8 frames per step

```
859         self.batch = 4 if device.type == "cuda" else 1
860         self.tile = None
861         self.step = 8
```

dev/engine.py · lines 873–880 · one model call for the whole batch, then finish each frame

```
873     def _process(self, frames):
874         x = frames_to_tensor(frames, self.device)
875         with torch.no_grad():
876             if self.tile:
877                 sr = tiled_forward(self._forward, x, self.tile, pad=40)
878             else:
879                 sr = self._forward(x)
880             return [self.finish(sr[j:j + 1], x[j:j + 1]) for j in range(len(frames))]
```

Why we used it

It gives more frames per second on the same GPU, which means fewer of your free GPU hours used.

How it works

`frames_to_tensor` stacks the frames into one tensor of shape `(batch, 3, height, width)`, and the model processes them together. If memory runs out, the batch halves automatically (concept 5).

What would go wrong without it

The GPU would be under-used, making the run slower. *If the batch is too big*, you run out of memory, which is why the batch is adjustable.

Alternatives and trade-offs

A bigger batch is faster up to a point, then memory runs out. The best size depends on the video resolution. The code starts at 4 and adapts down, but it never tries going *up*, which would be a possible improvement.

Interview prep

> **Q: Why batch frames?**
> A: GPUs do many calculations in parallel, so one call on four frames costs much less than four calls on one. I start at four and halve automatically if memory runs out.

Future use

Use it for **database inserts** (insert 1,000 rows in one query, not 1,000 queries), API calls (batch endpoints), sending emails, and ML inference servers.

23. Tiling with overlap ("halo" padding)

★★ Important · Category: performance / algorithms

What it is

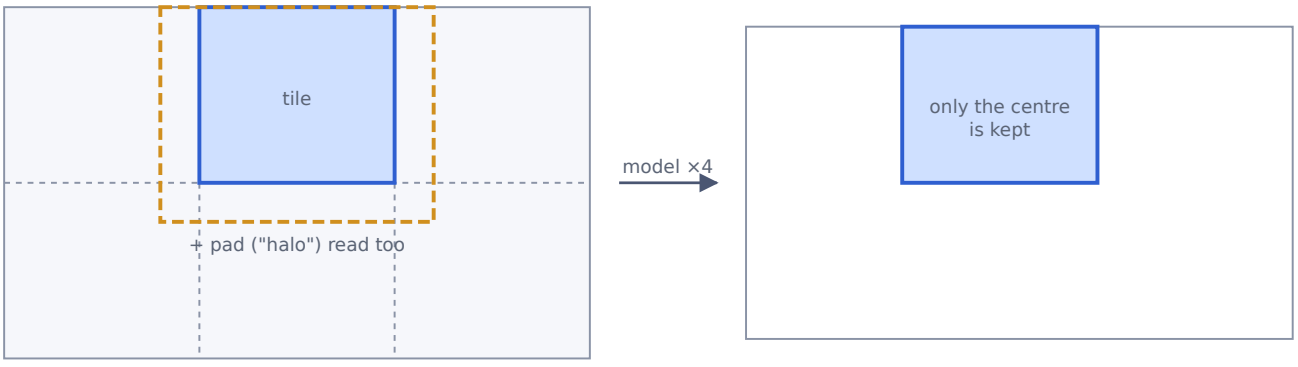
Analogy: painting a big wall in squares. If you paint each square only up to its edge, you get visible lines. So each time you paint a bit past the edge, and keep only the middle part.

Technically: when a frame is too big for GPU memory, split it into **tiles** (squares), process each one separately, and stitch them back together. Every pixel's result depends on the pixels around it, so each tile is read with extra **padding** (a "halo") and only the centre of the output is kept. That way the seams are invisible.

Where it is in your project

dev/engine.py · lines 823–838 · tiled_forward

```python
def tiled_forward(fn, x, tile, pad, scale=4, canvas_device=None):
    """Runs fn on overlapping squares of x and stitches the results (saves GPU memory)."""
    b, c, h, w = x.shape
    canvas = torch.empty(b, 3, h * scale, w * scale, dtype=torch.float16,
                         device=canvas_device or x.device)
    for y0 in range(0, h, tile):
        for x0 in range(0, w, tile):
            y1, x1 = min(y0 + tile, h), min(x0 + tile, w)
            py0, px0 = max(0, y0 - pad), max(0, x0 - pad)
            py1, px1 = min(h, y1 + pad), min(w, x1 + pad)
            out = fn(x[:, :, py0:py1, px0:px1])
            oy, ox = (y0 - py0) * scale, (x0 - px0) * scale
            canvas[:, :, y0 * scale:y1 * scale, x0 * scale:x1 * scale] = \
                out[:, :, oy:oy + (y1 - y0) * scale, ox:ox + (x1 - x0) * scale].to(canvas.dtype)
            del out
    return canvas
```



Input frame split into tiles **Stitched output canvas — no visible seams**

Figure 5 — Tiling with a halo (padding) so the edges of each tile are correct.

Why we used it

Inside the steady model, one 1080p frame becomes 64 channels × 4320 × 7680 numbers at 4× size. That is about 4 GB for a single layer, even in half precision, and several such layers are alive at once, which can go past the T4's 16 GB. Tiling makes any resolution possible, just slower.

How it works

1. Two loops walk over the frame in steps of `tile` pixels (`y0`, `x0`).

2. For each tile, a **padded** area is computed (`py0 … py1`), clamped to the image edges with `max` / `min`.
3. `fn(...)` upscales the padded piece.
4. The output position of the real tile inside the padded result is `(y0 - py0) * scale`.
5. Only that centre part is copied into the big `canvas` (stored in float16 to save memory).
6. The padding (40 px for fast, 48 px for steady) is at least the model's **receptive field**, meaning how far away a pixel can influence another. The test checks that tiled and non-tiled results differ by at most 2/255.

What would go wrong without it

Large videos would simply fail with out-of-memory. And tiling *without* a halo gives visible grid lines in the video.

Alternatives and trade-offs

Tiles are slower (the padding is computed twice) and, for the steady model, motion estimation inside a small tile is a bit worse. So tiling is only used **as a fallback**, not by default.

Interview prep

> **Q: How do you avoid seams when tiling?**
> A: I read each tile with padding bigger than the network's receptive field, process the padded tile, and keep only the centre. Each kept pixel then had all the context it would have had in the full image. My test confirms the difference is at most 2 out of 255.

Future use

Use it for **map tiles** (Google Maps), **huge images** (medical or satellite imagery), large matrix maths, and splitting big files into chunks for upload.

24. Mixed precision (fp16) with a safety net

★★ Important · Category: GPU performance

What it is

Analogy: writing prices as "£3.50" instead of "£3.4999999". The shorter form is precise enough and much quicker to write.

Technically: numbers in a computer have a **precision**. *fp32* (32-bit float) is precise. *fp16* (16-bit "half precision") uses half the memory and runs much faster on the T4's special "Tensor Cores", but it's less precise and can overflow. `torch.autocast` automatically uses fp16 for operations where it's safe, and keeps fp32 where it isn't. That mix is called **mixed precision**.

Where it is in your project

dev/engine.py · lines 863–871 · the fast model runs under autocast (only on the GPU)

```
863     def _forward(self, x):
864         pad = 8
865         h, w = x.shape[-2:]
866         mode = "reflect" if min(h, w) > pad else "replicate"
867         xp = F.pad(x, (pad, pad, pad, pad), mode=mode)
868         with torch.no_grad(), torch.autocast(device_type=self.device.type, dtype=torch.float16,
869                                              enabled=self.half):
870             out = self.net(xp)
871         return out[:, :, pad * 4:pad * 4 + h * 4, pad * 4:pad * 4 + w * 4]
```

dev/engine.py · lines 798–801 · the Finisher checks every output for NaN/infinity

```
798     def __call__(self, sr4, lr):
799         sr = sr4.to(self.device).float()
800         if not torch.isfinite(sr).all():
801             raise NonFiniteOutput()
```

dev/engine.py · lines 900–904 · if that happens: switch to full precision and redo

```
900             except NonFiniteOutput:
901                 if not self.half:
902                     raise UpscalerError("The model produced broken pixels. Please report this video.")
903                 print("\n(Half precision gave bad pixels - switching to full precision. Automatic.)")
904                 self.half = False
```

Why we used it

Roughly twice the speed and half the memory on a T4, which means more frames per free GPU hour and fewer out-of-memory fallbacks.

How it works

1. `self.half = S.USE_HALF_PRECISION and device.type == "cuda"`: it's only enabled on a GPU (CPUs don't benefit).
2. `with torch.autocast(..., dtype=torch.float16, enabled=self.half):` runs the model's maths in fp16 where it's safe.
3. The output is converted back to fp32 in the `Finisher` (`.float()`) before resizing and blending.
4. `torch.isfinite(sr).all()` checks for broken values. If any are found, `NonFiniteOutput` is raised, the engine sets `self.half = False`, and **the same frames are processed again** in fp32.

5. In `flow_warp`, the motion-warping step is always done in fp32 (`x.float()`), because pixel coordinates like 1919.5 can't be stored exactly in fp16.

What would go wrong without it

Without fp16 you get half the speed. **With fp16 but no safety net**, a rare overflow could produce black or garbage frames in the final video without anyone noticing.

Alternatives and trade-offs

bfloat16 is safer, but the T4 doesn't support it well. *INT8 quantisation* is even faster, but needs calibration and can reduce quality. *Weakness:* the NaN check only catches *broken* numbers, not small quality losses. fp16 inference is generally fine for these models, though.

Interview prep

> **Q: What is mixed precision and what are the risks?**
> A: Running most maths in 16-bit floats for speed and memory, while keeping sensitive parts in 32-bit. The risk is overflow or loss of precision. I use autocast, keep the coordinate maths in fp32, and check every output for NaN or infinity. If one appears, I automatically redo it in fp32.

Future use

Use it for training or serving ML models on GPUs, and on mobile/edge AI. The general idea, "use the cheaper format, verify the result, fall back when needed", applies to image compression and approximate algorithms too.

25. GPU device handling and memory management

★★ Important · Category: machine learning / resources

What it is

Analogy: the GPU is a separate workshop with its own small storeroom (16 GB). Materials have to be carried there (`.to(device)`), and the storeroom has to be cleared regularly.

Technically: PyTorch tensors live on a **device**: `cpu` or `cuda:0` (the first NVIDIA GPU). Data must be moved to the GPU before GPU maths can use it. PyTorch keeps freed GPU memory reserved for reuse, and `torch.cuda.empty_cache()` gives it back. **Inference mode** (`torch.no_grad()` + `model.eval()`) stops PyTorch from storing the extra data that training would need.

Where it is in your project

dev/engine.py · lines 1120–1129 · get_device with a friendly "turn on the GPU" message

```
1120 def get_device(allow_cpu=False):
1121     if torch.cuda.is_available():
1122         return torch.device("cuda:0")
1123     if allow_cpu:
1124         return torch.device("cpu")
1125     raise UpscalerError(
1126         "No GPU found. The GPU is switched off.\n"
1127         "Fix: in the right-hand panel of the notebook open 'Session options' (or 'Settings'),\n"
1128         "set ACCELERATOR to 'GPU T4 x2', confirm, then run all cells again from the top.\n"
1129         "(If you don't see the option, verify your phone number in your Kaggle account settings.)")
```

dev/engine.py · lines 774–777 · free_memory

```
774 def free_memory():
775     gc.collect()
776     if torch.cuda.is_available():
777         torch.cuda.empty_cache()
```

dev/engine.py · lines 973–974 · keep the steady model's features on the GPU only if they're under 2.5 GB

```
973         feat_bytes = keep_count * 64 * h * w * (2 if self.half else 4)
974         on_gpu = self.device.type == "cuda" and feat_bytes < 2.5 * 1024 ** 3
```

Why we used it

The same code needs to run on Kaggle's GPU and on a normal CPU for the tests (`allow_cpu=True`). And GPU memory is the scarcest resource in the whole project.

How it works

1. `get_device()` returns `cuda:0` if a GPU exists. Otherwise it raises a clear error, unless CPU use is allowed for tests.
2. Models are moved once (`self.net.eval().to(device)`), and each batch of frames is moved when it's processed (`frames_to_tensor`).
3. Results come back to normal memory with `.cpu().numpy()` before being written.
4. `free_memory()` runs Python's garbage collector (`gc.collect()`, which deletes unreachable objects) and then empties PyTorch's GPU cache. It's called after out-of-memory errors, when switching models, and between tiles.

5. The steady model estimates how big its stored features will be (`keep_count × 64 × h × w × bytes`) and keeps them in **CPU memory** if they're too large. This is called **offloading**.
6. `del out` releases big tensors as soon as they're no longer needed.

What would go wrong without it

Crashes on machines without a GPU, a confusing "CUDA not available" error for users who forgot to enable it, and out-of-memory errors that could have been avoided.

Alternatives and trade-offs

Using both T4s on Kaggle ("T4 x2") could almost double the speed. Your code uses only one, which is a good improvement to mention. Calling `empty_cache()` too often slows things down slightly, so it's only used at key moments.

Interview prep

> **Q: How do you manage GPU memory?**
> A: Inference mode, so nothing is stored for training. Half precision. Freeing the cache after errors and between tiles. Offloading large intermediate features to CPU memory. And the automatic fallback to smaller batches and tiles. The code also runs on the CPU for tests.

Future use

Any ML project. More generally, managing a **limited resource** (connections, memory, file handles): measure it, release it early, and have a fallback plan.

26. Closures and callbacks (the `emit` function)

★★ Important · Category: Python / functional programming

What it is

Analogy: you give a delivery driver your phone number and say "call me when each parcel arrives". The driver doesn't need to know what you'll do with each parcel. They just call.

Technically: a **callback** is a function you pass to other code, which it calls when something happens. A **closure** is a function defined inside another function that **remembers the variables around it** (here `results`, `x`, `canvases` and the tile position), even when it's called from somewhere else.

Where it is in your project

dev/engine.py · lines 976–979 · emit stores each finished frame in results

```
976            if not self.tile:
977                def emit(k, sr):
978                    results[k] = self.finish(sr, x[keep_from + k:keep_from + k + 1])
979                self.net.run(x, keep_from, keep_count, self.passes, emit, on_gpu)
```

dev/engine.py · lines 990–994 · in tiled mode emit pastes each tile into the right place

```
990                        def emit(k, sr):
991                            canvases[k][:, :, y0 * 4:y1 * 4, x0 * 4:x1 * 4] = \
992                                sr[:, :, oy:oy + (y1 - y0) * 4, ox:ox + (x1 - x0) * 4].to("cpu",
torch.float16)
993                        self.net.run(x[:, :, py0:py1, px0:px1].contiguous(), keep_from, keep_count,
994                                     self.passes, emit, on_gpu)
```

dev/engine.py · lines 754–755 · the model calls emit as soon as a frame is ready

```
754                emit(i - keep_from, out)
755                del out
```

Why we used it

The model produces frames **one by one**, deep inside a loop. If it collected all 30 full-size outputs first, memory could run out. With a callback, each frame is **finished immediately** (resized, blended, converted to bytes) and the big 4× tensor is freed. The *same* model code also works for both "whole frame" and "tiled" modes, because only the callback changes.

How it works

1. `_window` creates `results = [None] * keep_count`.
2. It defines `emit(k, sr)`, which uses `results`, `x` and `keep_from` from the surrounding function (that's the closure).
3. It passes `emit` into `self.net.run(...)`.
4. Inside the model loop, each time an output frame is ready: `emit(i - keep_from, out)`, then `del out`.
5. In tiled mode, a *different* `emit` is defined for each tile. It remembers that tile's `y0, x0, oy, ox` and pastes the result into a CPU canvas.

What would go wrong without it

Either much higher memory use (storing every output first), or two copies of the complex model loop, one per mode.

Alternatives and trade-offs

A **generator** (`yield out`) is the other Pythonic choice: the caller loops over results. It's equally good. Callbacks were simpler here because the tiled mode needs to put results somewhere *different*. *Gotcha:* closures in loops capture **variables, not values**. This works because `emit` is used immediately inside the same loop iteration. If it were stored and called later, all copies would see the last tile's values.

Interview prep

> **Q: What's a closure? Where do you use one?**
>
> A: A function that remembers variables from where it was defined. My model loop calls an emit callback for each finished frame. The callback is a closure over the results list, or over the tile's position in tiled mode. So the same model code serves both modes, and each big frame is freed immediately.

Future use

Use callbacks for **event handlers** (button clicks in JavaScript, Android `onClick`), progress reporting, async completion handlers, and plugin hooks. Closures are used everywhere in JavaScript and in React hooks.

27. Recursion as divide-and-conquer

★★ Important · Category: algorithms

What it is

Analogy: a pizza that's too big for the oven. Cut it in half. Still too big? Cut those halves in half, and so on.

Technically: recursion means a function calling itself on a smaller part of the problem. **Divide and conquer** means splitting a problem into smaller pieces until each one is easy.

Where it is in your project

dev/engine.py · lines 1004–1010 · too many frames for one chunk → run() on each piece

```
1004        if len(core) > self.chunk:
1005            out = []
1006            for s in range(0, len(core), self.chunk):
1007                b = (list(before) + list(core[:s]))[-ctx:]
1008                a = (list(core[s + self.chunk:]) + after)[:ctx]
1009                out += self.run(b, core[s:s + self.chunk], a)
1010            return out
```

dev/engine.py · lines 1018–1033 · out of memory → make chunks smaller (or tiles) → call run() again

```
1018        except Exception as e:
1019            if not is_memory_error(e):
1020                raise
1021            free_memory()
1022            if len(core) > 6:
1023                self.chunk = max(6, len(core) // 2)
1024                print(f"\n(GPU memory is tight - now doing {self.chunk} frames at a time.
Automatic.)")
1025            else:
1026                smaller = [t for t in TILE_STEPS if self.tile is None or t < self.tile]
1027                if not smaller:
1028                    raise UpscalerError("Ran out of GPU memory even with the smallest settings. "
1029                                        "Try a smaller SCALE, a lower-resolution video, or MODEL =
\"fast\".")
1030                self.tile = smaller[0]
1031                print(f"\n(GPU memory is tight - switching to tiles of {self.tile} px. Automatic.)")
1032        free_memory()
1033        return self.run(before, core, after)
```

Why we used it

`SteadyEngine.run` might be given more frames than fit in memory. Splitting them up and calling itself again keeps the logic short, and each piece automatically gets correct before/after context.

How it works

1. **Base case** (where recursion stops): `len(core) <= self.chunk` and processing succeeds, so it returns the results.
2. **Recursive case:** if there are too many frames, loop over pieces of `chunk` frames, build each piece's context from neighbouring frames, and call `self.run(...)` on it.
3. On a memory error, it lowers `self.chunk` (or switches on tiles) and calls `self.run(...)` again on the **same** frames. That call then splits them.
4. Each retry changes something (smaller chunk, smaller tile, or fp32), and there's a floor (6 frames, 128-pixel tiles) where it gives up with an error. That guarantees it **can't loop forever**.

What would go wrong without it

Without a guaranteed base case, recursion can loop forever or overflow the stack. That's why every retry must make progress.

Alternatives and trade-offs

A `while` loop with a queue of pending pieces would do the same thing without recursion. It's more code, but has no depth limit. Here the depth is tiny (a few levels), so recursion is clearer.

Interview prep

> **Q: How do you make sure your recursion ends?**
> A: Every recursive call either works on fewer frames, or after changing a setting that only gets smaller: chunk size, then tile size. Both have a minimum, and when it's reached I raise an error instead of recursing again.

Future use

Use it for tree structures (folders, comments with replies, menus), sorting algorithms (merge sort, quicksort), parsing nested data like JSON, and splitting work until it fits.

28. Weight interpolation (adjustable denoise strength)

★★ Important · Category: machine learning

What it is

Analogy: mixing hot and cold water to get exactly the temperature you want.

Technically: Real-ESRGAN provides **two trained versions** of the same small network: one that removes noise strongly (`general-x4v3`) and one that keeps noise (`general-wdn-x4v3`). Because they have identical shapes, you can **blend their weights** number by number: `mixed = d·A + (1−d)·B`. The result behaves in between. The official project calls this **DNI** (Deep Network Interpolation).

Where it is in your project

dev/engine.py · lines 849–857 · mix the two models with DENOISE_STRENGTH

```
849         a = torch_load(get_weights("general"))
850         b = torch_load(get_weights("general_wdn"))
851         a = a.get("params_ema", a.get("params", a))
852         b = b.get("params_ema", b.get("params", b))
853         d = float(S.DENOISE_STRENGTH)
854         # Mix the "strong denoise" and "keep noise" models (official Real-ESRGAN trick)
855         mixed = {k: d * a[k].float() + (1 - d) * b[k].float() for k in a}
856         self.net = SRVGGNetCompact()
857         self.net.load_state_dict(mixed, strict=True)
```

Why we used it

Your number one goal was a **natural look**. Too much denoising makes skin plastic, and too little keeps ugly noise. A single 0–1 slider gives fine control, without training anything and without running two models.

How it works

1. Load both weight files (`torch_load`) and pick the `params_ema` or `params` dictionary from each.
2. For every layer name `k`: `d * a[k] + (1 - d) * b[k]`.
3. Load the mixed dictionary into one network with `strict=True`.
4. The cost is the same as running one model. The mixing happens only once, when the engine is built.

What would go wrong without it

You'd only have the two extremes ("plastic" or "noisy"), or you'd have to run both models and blend the *pictures*, which takes twice the time.

Alternatives and trade-offs

Blending output **images** (which `ORIGINAL_BLEND` does with the original video) is a different, complementary trick. Weight interpolation only works when the two models share the same architecture and were fine-tuned from a common starting point. Otherwise the mix produces nonsense.

Interview prep

> **Q: How does your denoise slider work?**
>
> A: Real-ESRGAN ships a strong-denoise model and a keep-noise model with identical layers. I interpolate their weights linearly by the slider value and load the result into one network. So I get a continuous control with no extra running cost.

Future use

This is the same idea as model "soups" or merges in Stable Diffusion and LLM communities, and as style-strength sliders. More generally: when you have two good extremes, look for a parameter that blends between them.

29. Residual connections (and PixelShuffle)

★★ Important · Category: machine learning

What it is

Analogy: instead of repainting a whole picture from scratch, an artist starts with a photocopy and only paints the *corrections*.

Technically: a **residual connection** adds a layer's input to its output: `output = x + f(x)`. The layers only need to learn the *difference* (the "residual"), which makes deep networks much easier to train and more stable. **PixelShuffle** is a clever way to make a picture bigger: the network outputs 16 times as many channels, and PixelShuffle rearranges them into a 4× wider and 4× taller image.

Where it is in your project

dev/engine.py · lines 573–581 · residual block: x + conv(relu(conv(x)))

```
573 class ResidualBlockNoBN(nn.Module):
574     def __init__(self, ch=64):
575         super().__init__()
576         self.conv1 = nn.Conv2d(ch, ch, 3, 1, 1)
577         self.conv2 = nn.Conv2d(ch, ch, 3, 1, 1)
578         self.relu = nn.ReLU(inplace=True)
579 
580     def forward(self, x):
581         return x + self.conv2(self.relu(self.conv1(x)))
```

dev/engine.py · lines 565–570 · the whole fast model is a residual on top of a simple enlargement

```
565     def forward(self, x):
566         out = x
567         for layer in self.body:
568             out = layer(out)
569         out = self.upsampler(out)
570         return out + F.interpolate(x, scale_factor=self.upscale, mode="nearest")
```

dev/engine.py · lines 595–602 · PixelShufflePack: conv then pixel_shuffle

```
595 class PixelShufflePack(nn.Module):
596     def __init__(self, in_ch, out_ch, scale, kernel=3):
597         super().__init__()
598         self.scale = scale
599         self.upsample_conv = nn.Conv2d(in_ch, out_ch * scale * scale, kernel, padding=(kernel - 1) //
2)
600 
601     def forward(self, x):
602         return F.pixel_shuffle(self.upsample_conv(x), self.scale)
```

Why we used it

These come from the official model designs. Your code had to reproduce them exactly so the downloaded weights would fit. Understanding them lets you explain *why* the models work.

How it works

- In `ResidualBlockNoBN.forward`: `x + self.conv2(self.relu(self.conv1(x)))`. If the convolutions learn nothing useful, the block simply passes `x` through, so it can never make things much worse.

- In `SRVGGNetCompact.forward`: the result is `network_output + nearest_neighbour_enlargement(x)`, so the network only adds the missing detail.
- PixelShuffle: 48 channels (3 colours × 4 × 4) at low resolution are rearranged into 3 channels at 4× the resolution. It's cheaper than enlarging first and computing at full size.

What would go wrong without it

Deep networks without residual connections are hard to train and tend to lose detail. If your copy left them out, the official weights wouldn't fit, or would produce garbage.

Alternatives and trade-offs

"Transposed convolution" is another way to enlarge, but it often creates checkerboard artefacts, which PixelShuffle avoids. Interpolation followed by a convolution is simpler, but slower at high resolution.

Interview prep

> **Q: What is a residual connection?**
> A: Adding a block's input to its output, so the block only learns the change. It makes deep networks stable to train. In my fast model, even the whole network is residual: it adds detail on top of a simple enlargement of the input.

Future use

Residual connections are in nearly every modern network: ResNet for images, and every Transformer and LLM. PixelShuffle appears in super-resolution and image generation models.

30. How the steady model sees motion: optical flow, warping and two-way propagation

★★ Important · Category: machine learning / video

What it is

Analogy: to restore one blurry frame of a car driving past, look at the frames just before and after. The car is in a slightly different place in each, so first work out *how it moved*, slide those frames to line up, and then combine what you know.

Technically: optical flow is a map of how far every pixel moved between two frames. **Warping** uses that map to shift an image (or the network's internal "memory", called *features*) so it lines up with the current frame. **SPyNet** estimates the flow coarse-to-fine: first on tiny versions of the frames, then on bigger ones. **Bidirectional propagation** runs one pass backward in time and one forward, so each frame gets information from both its past and its future.

Where it is in your project

dev/engine.py · lines 627–637 · flow_warp: shift pixels along the flow

```
627 def flow_warp(x, flow, padding_mode="zeros"):
628     """Moves the pixels of x along the motion 'flow' (N,H,W,2)."""
629     _, _, h, w = x.shape
630     gy, gx = torch.meshgrid(torch.arange(h, device=x.device, dtype=torch.float32),
631                             torch.arange(w, device=x.device, dtype=torch.float32), indexing="ij")
632     grid = torch.stack((gx, gy), 2) + flow.float()
633     gfx = 2.0 * grid[..., 0] / max(w - 1, 1) - 1.0
634     gfy = 2.0 * grid[..., 1] / max(h - 1, 1) - 1.0
635     out = F.grid_sample(x.float(), torch.stack((gfx, gfy), dim=3), mode="bilinear",
636                         padding_mode=padding_mode, align_corners=True)
637     return out.to(x.dtype)
```

dev/engine.py · lines 649–664 · SPyNet: an image pyramid, coarse to fine

```
649     def compute_flow(self, ref, supp):
650         n, _, h, w = ref.shape
651         ref = [(ref - self.mean) / self.std]
652         supp = [(supp - self.mean) / self.std]
653         for _ in range(5):
654             ref.append(F.avg_pool2d(ref[-1], 2, 2, count_include_pad=False))
655             supp.append(F.avg_pool2d(supp[-1], 2, 2, count_include_pad=False))
656         ref, supp = ref[::-1], supp[::-1]
657         flow = ref[0].new_zeros(n, 2, h // 32, w // 32)
658         for level in range(len(ref)):
659             flow_up = flow if level == 0 else F.interpolate(
660                 flow, scale_factor=2, mode="bilinear", align_corners=True) * 2.0
661             flow = flow_up + self.basic_module[level](torch.cat([
662                 ref[level], flow_warp(supp[level], flow_up.permute(0, 2, 3, 1),
padding_mode="border"),
663                 flow_up.to(ref[level].dtype)], 1))
664         return flow
```

dev/engine.py · lines 729–737 · backward pass (future → past)

```
        back = {}
        feat = lqs.new_zeros(1, bv.mid_channels, h, w)
        for i in range(t - 1, keep_from - 1, -1):          # look backwards in time
            if i < t - 1:
                flow = bv.spynet(lqs[i:i + 1], lqs[i + 1:i + 2])
                feat = flow_warp(feat, flow.permute(0, 2, 3, 1))
            feat = bv.backward_resblocks(torch.cat([lqs[i:i + 1], feat.to(lqs.dtype)], 1))
            if i <= last:
                back[i] = feat.to(store, non_blocking=True)
```

dev/engine.py · lines 739–754 · forward pass (past → future) + combine + upscale

```
        feat = torch.zeros_like(feat)
        for i in range(0, last + 1):                        # look forwards in time
            if i > 0:
                flow = bv.spynet(lqs[i:i + 1], lqs[i - 1:i])
                feat = flow_warp(feat, flow.permute(0, 2, 3, 1))
            feat = bv.forward_resblocks(torch.cat([lqs[i:i + 1], feat.to(lqs.dtype)], 1))
            if i >= keep_from:
                out = torch.cat([back.pop(i).to(feat.device), feat], 1)
                out = bv.lrelu(bv.fusion(out))
                out = bv.lrelu(bv.upsample1(out))
                out = bv.lrelu(bv.upsample2(out))
                out = bv.lrelu(bv.conv_hr(out))
                out = bv.conv_last(out)
                out = out + F.interpolate(lqs[i:i + 1], scale_factor=4, mode="bilinear",
                                          align_corners=False).to(out.dtype)
                emit(i - keep_from, out)
```

Why we used it

This is what makes "steady" steady. A frame-by-frame model invents slightly different detail in each frame, which shows up as shimmering. Sharing aligned information across frames makes the invented detail **consistent over time**.

How it works

1. **Cleaning:** a first sub-network removes noise and compression blocks (`clean`). How many passes it does is decided **once per video** (`calibrate`), so the look never jumps mid-video.
2. **Backward pass:** start from the last frame of the window. For each earlier frame: estimate the flow to the next frame, warp the "memory" with it, then update the memory with the current frame (`backward_resblocks`). Store it.
3. **Forward pass:** the same from the first frame onwards (`forward_resblocks`).
4. For each frame to output, **combine** both memories (`fusion`), upscale twice by 2 (`upsample1`, `upsample2`), and add a simple enlargement of the input (the residual idea again).
5. `grid_sample` does the actual shifting. The coordinates are converted to the range −1…1 that it expects.

What would go wrong without it

Flicker and shimmer on fine textures (hair, leaves, fabric). Without alignment, combining frames would create ghosting: double images of anything moving.

Alternatives and trade-offs

Newer video models (BasicVSR++, transformer-based models) are better but heavier. Deformable convolutions are another alignment method, but they need compiled extensions. *Cost:* RealBasicVSR is several times slower than the fast model, and on clean videos it can look a bit

"painted". That's why you can choose between the two.

Interview prep

> **Q: How does the steady model reduce flicker?**
> A: It estimates optical flow between neighbouring frames, warps its internal features to line them up, and propagates information both backwards and forwards in time. Each output frame then uses consistent details from its neighbours, instead of inventing new ones each time.

Future use

Optical flow is used in video stabilisation, frame interpolation (slow motion), object tracking, video compression and self-driving cars. Two-way processing also appears in bidirectional RNNs and BERT for text.

31. Reading video metadata with ffprobe (defensive parsing)

★★ Important · Category: video / data parsing

What it is

Analogy: reading the label on a box before opening it: what's inside, how big it is, and which way up it goes.

Technically: ffprobe (part of ffmpeg) prints a video's details as **JSON**. Your code parses it **defensively**, meaning it assumes any field might be missing, empty or odd, and always has a fallback.

Where it is in your project

dev/engine.py · lines 213–224 · run ffprobe, check for errors, pick the real video stream

```
213     r = subprocess.run(["ffprobe", "-v", "error", "-print_format", "json",
214                         "-show_streams", "-show_format", path], capture_output=True, text=True)
215     if r.returncode != 0:
216         raise UpscalerError(f"Could not read '{name}'. It may be damaged or not a real video.\n"
217                             f"ffprobe said: {r.stderr.strip()[-400:]}")
218     data = json.loads(r.stdout or "{}")
219     streams = data.get("streams", [])
220     vids = [s for s in streams if s.get("codec_type") == "video"
221             and not s.get("disposition", {}).get("attached_pic")]
222     if not vids:
223         raise UpscalerError(f"'{name}' has no video picture inside it (maybe it is audio only).")
224     vs = vids[0]
```

dev/engine.py · lines 239–248 · choose a trustworthy frame rate

```
239     r_rate, a_rate = parse_rate(vs.get("r_frame_rate")), parse_rate(vs.get("avg_frame_rate"))
240     if r_rate and r_rate <= 120 and (not a_rate or abs(float(r_rate) - float(a_rate)) / float(r_rate)
< 0.01):
241         fps = r_rate
242     elif a_rate and a_rate <= 240:
243         fps = a_rate
244     elif r_rate and r_rate <= 240:
245         fps = r_rate
246     else:
247         fps = Fraction(30, 1)
248     fps = fps.limit_denominator(1001)
```

dev/engine.py · lines 263–272 · choose the colour matrix (with a sensible guess if untagged)

```
263     space = (vs.get("color_space") or "").lower()
264     transfer = (vs.get("color_transfer") or "").lower()
265     if space in ("bt709",):
266         matrix = "bt709"
267     elif space in ("smpte170m", "bt470bg", "fcc"):
268         matrix = "bt601"
269     elif space.startswith("bt2020"):
270         matrix = "bt2020"
271     else:
272         matrix = "bt709" if w * h >= 1280 * 720 else "bt601"
```

Why we used it

Everything later depends on these facts: frame size (raw frames are exactly width × height × 3 bytes), frame rate (output timing and audio sync), rotation, HDR, and whether there's audio. Real-world files, especially from phones, are messy.

How it works

1. `-print_format json -show_streams -show_format`, then `json.loads` turns the text into Python dictionaries.
2. It ignores "attached pictures" (album-cover images that some files store as a video stream).
3. `.get(key, default)` everywhere, so a missing field never crashes the code.
4. Frame rate: `r_frame_rate` ("base rate") and `avg_frame_rate` are both read. It uses the one that makes sense, defaults to 30, and stores it as an exact fraction (`30000/1001` = 29.97).
5. Duration: tries the stream's value, then the container's.
6. Colour matrix: uses the tag if present. If untagged, it guesses BT.709 for HD sizes and BT.601 for smaller ones (the common convention).

What would go wrong without it

Wrong frame size means garbled pictures. A wrong frame rate means the audio drifts out of sync. A wrong colour matrix shifts the colours slightly. Missing fields would crash the code on some phone videos.

Alternatives and trade-offs

Python libraries like `pymediainfo` or PyAV give similar information. ffprobe is already installed alongside ffmpeg and is the reference tool. *Weakness:* the estimated frame count can be slightly off for variable-frame-rate videos, but it's only used for the progress bar and ETA.

Interview prep

> **Q: How do you handle messy input files?**
> A: I read their metadata as JSON with ffprobe and parse defensively: every field has a fallback, I sanity-check the frame rates, and I handle rotation and HDR. Anything truly broken raises a clear error naming the file.

Future use

Parsing **any external data**: API responses, uploaded files, CSVs and webhooks. Always use `.get()` with defaults, validate, and never assume a field exists.

32. H.264 encoding settings

★★ Important · Category: video

What it is

Analogy: packing a suitcase. You can fold carefully (smaller, slower) or stuff it in (faster, bigger). The goal is to fit everything without crushing your clothes (losing quality).

Technically: H.264 is the most widely supported video compression format. **libx264** is the encoder. Key settings: **CRF** (Constant Rate Factor, a *quality target*: lower means better and bigger, ~17 looks visually lossless), **preset** (speed versus compression efficiency), **tune** (hints like `film` or `grain`), and **yuv420p** (the pixel format every player supports).

Where it is in your project

dev/engine.py · lines 420–427 · the encoder command

```
420         tune = "grain" if S.FILM_GRAIN >= 0.02 else "film"
421         cmd = ["ffmpeg", "-hide_banner", "-nostdin", "-loglevel", "error", "-y",
422                "-f", "rawvideo", "-pix_fmt", "rgb24", "-s", f"{width}x{height}",
423                "-framerate", fps_str, "-i", "pipe:0",
424                "-vf",
"scale=out_color_matrix=bt709:out_range=tv:flags=bicubic+accurate_rnd+full_chroma_int,format=yuv420p",
425                "-c:v", "libx264", "-preset", S.ENCODER_SPEED, "-crf", str(S.VIDEO_QUALITY), "-tune",
tune,
426                "-colorspace", "bt709", "-color_primaries", "bt709", "-color_trc", "bt709", "-
color_range", "tv",
427                "-movflags", "+faststart", path]
```

dev/engine.py · lines 283–286 · output sizes are always even numbers

```
283 def output_size(info, scale):
284     def even(x):
285         return max(2, int(round(x / 2.0)) * 2)
286     return even(info["width"] * scale), even(info["height"] * scale)
```

Why we used it

The result has to look natural *and* play everywhere (phone, TV, browser). Compression can smear fine grain and skin texture, so the settings protect detail.

How it works

- `-crf 17` by default (`VIDEO_QUALITY`) keeps quality constant. Bitrate goes up for complex scenes and down for simple ones.
- `-preset medium` (`ENCODER_SPEED`): slower presets give smaller files at the same quality.
- `-tune film`, or `-tune grain` when `FILM_GRAIN ≥ 0.02`, tells the encoder to keep fine texture instead of smoothing it away.
- `yuv420p` stores colour at a quarter of the resolution. That's what all players expect, and it's why width and height **must be even**, which `output_size` guarantees.
- `-movflags +faststart` puts the file's index at the start, so the video can start playing before it has fully downloaded.
- Colour tags (`-colorspace bt709` …) tell players how to show the colours correctly.

What would go wrong without it

Blocky or over-smoothed output (with a low quality setting), huge files (with a very low CRF), files that won't play on phones (other pixel formats), or an encoder error on odd sizes like 1695×955.

Alternatives and trade-offs

H.265/HEVC and **AV1** give smaller files at the same quality, but encode slower and are less universally supported. NVIDIA's hardware encoder (NVENC) is much faster, but its quality per size is lower. H.264 was the right choice for "plays everywhere, good quality".

Interview prep

> **Q: What is CRF and why use it instead of a bitrate?**
> A: CRF targets a constant visual quality and lets the bitrate vary with scene complexity. That suits a quality-first tool. A fixed bitrate would waste bits on simple scenes and starve complex ones.

Future use

Any app that handles video: uploads, thumbnails, streaming, screen recording. The same trade-offs (quality, size, speed, compatibility) apply to images (JPEG, WebP, AVIF) and audio too.

33. Segments, concatenation and audio muxing

★★ Important · Category: video

What it is

Analogy: filming a movie in scenes, then editing the scenes together and adding the soundtrack at the end.

Technically: the video is written as many short MP4 **segments**. At the end, ffmpeg's **concat demuxer** (a reader that treats a list of files as one) joins them **without re-encoding**, and **muxing** (combining streams into one file) adds the original audio, copied unchanged where possible.

Where it is in your project

dev/engine.py · lines 1503–1526 · assemble

```
1503 def assemble(work_dir, progress, info, final_path):
1504     segs = [os.path.join(work_dir, s["file"]) for s in progress["segments"]]
1505     lst = os.path.join(work_dir, "segments.txt")
1506     with open(lst, "w") as f:
1507         for s in segs:
1508             f.write(f"file '{os.path.basename(s)}'\n")
1509     tmp = final_path + ".partial.mp4"
1510     base = ["ffmpeg", "-hide_banner", "-nostdin", "-loglevel", "error", "-y",
1511             "-f", "concat", "-safe", "0", "-i", lst]
1512     attempts = []
1513     if info["has_audio"]:
1514         audio_in = ["-i", info["path"], "-map", "0:v:0", "-map", "1:a?", "-map_metadata", "1"]
1515         attempts.append(audio_in + ["-c:v", "copy", "-c:a", "copy"])
1516         attempts.append(audio_in + ["-c:v", "copy", "-c:a", "aac", "-b:a", "256k"])
1517     attempts.append(["-map", "0:v:0", "-c:v", "copy"])
1518     err = ""
1519     for i, extra in enumerate(attempts):
1520         r = subprocess.run(base + extra + ["-movflags", "+faststart", tmp], capture_output=True,
text=True)
1521         if r.returncode == 0 and os.path.exists(tmp) and os.path.getsize(tmp) > 0:
1522             if info["has_audio"] and i == len(attempts) - 1:
1523                 print("⚠️  The sound could not be added - the video was saved WITHOUT sound.")
1524             os.replace(tmp, final_path)
1525             return
1526         err = r.stderr
```

Why we used it

Segments are what make checkpointing possible (concept 3). Copying instead of re-encoding means joining is fast and loses no quality. Taking the audio from the original file keeps it perfect and in sync.

How it works

1. Write `segments.txt` with lines like `file 'seg_00000.mp4'`.
2. `-f concat -safe 0 -i segments.txt` reads them as one continuous video.
3. `-i original` is the second input, and `-map 0:v:0 -map 1:a?` takes the video from the parts and the audio from the original. The `?` means "if there is any".
4. `-c:v copy -c:a copy` means no re-encoding. If the audio format isn't allowed in MP4, the next attempt converts it to AAC. The last resort is video only, with a warning.
5. `-map_metadata 1` keeps the original's metadata (such as the creation date).
6. It writes to `.partial.mp4`, then renames (atomic, concept 4).

What would go wrong without it

Re-encoding everything at the end would waste time and quality. Without audio muxing, the output would be silent.

Alternatives and trade-offs

Encoding one big file in a single go is simpler, but can't resume. MPEG-TS segments join even more reliably, but MP4 parts were tested to work frame-perfectly here. *Weakness:* the concat step needs free disk space equal to all the parts, which is why the code checks for it first.

Interview prep

> **Q: How do you rebuild the final video with the original sound?**
> A: The upscaled video is written as short segments. At the end I concatenate them with ffmpeg's concat demuxer, copying the streams so nothing is re-encoded, and map the audio stream from the original file. If the audio codec can't go into MP4, I fall back to AAC.

Future use

Use it for video editing tools, livestream recording (HLS streaming is literally small segments plus a playlist), and splitting any big job into joinable pieces (for example, multipart uploads to cloud storage).

34. Colour spaces: RGB vs YUV, BT.601 vs BT.709

★★ Important · Category: video / image processing

What it is

Analogy: one colour described in two languages. Converting between languages needs the right dictionary, and the wrong one gives a slightly different colour.

Technically: AI models work on **RGB** (red, green, blue). Videos store **YUV** (brightness plus two colour-difference channels). The maths to convert between them is a **colour matrix**: **BT.601** for old/SD video, **BT.709** for HD. "TV range" means brightness values go from 16 to 235 instead of 0 to 255.

Where it is in your project

dev/engine.py · lines 305–306 · decoding: use the video's own matrix to get RGB

```
305         vf += [f"scale=in_color_matrix={info['matrix']}:flags=bicubic+accurate_rnd+full_chroma_int",
306                "format=rgb24"]
```

dev/engine.py · lines 424–426 · encoding: convert with BT.709 and tag the file as BT.709

```
424                 "-vf",
"scale=out_color_matrix=bt709:out_range=tv:flags=bicubic+accurate_rnd+full_chroma_int,format=yuv420p",
425                 "-c:v", "libx264", "-preset", S.ENCODER_SPEED, "-crf", str(S.VIDEO_QUALITY), "-tune",
tune,
426                 "-colorspace", "bt709", "-color_primaries", "bt709", "-color_trc", "bt709", "-
color_range", "tv",
```

Why we used it

ffmpeg's default RGB→YUV conversion uses BT.601, while HD players expect BT.709. Mixing them up gives a subtle but real **colour shift**: skin slightly too green or too pink. For a tool whose goal is "natural", that matters. In the test, the measured average shift was at most about 1 level out of 255 (the test fails above 3).

How it works

1. Decoding: `scale=in_color_matrix=<matrix>` converts using the matrix ffprobe found (or guessed).
2. Encoding: `scale=out_color_matrix=bt709:out_range=tv` converts RGB to YUV correctly for HD.
3. Tags (`-colorspace bt709 -color_primaries bt709 -color_trc bt709 -color_range tv`) tell players which "dictionary" to use when showing it.
4. `accurate_rnd+full_chroma_int` are flags for more accurate rounding and colour handling.

What would go wrong without it

Slightly wrong colours in every output video. It's a classic hidden bug that most hobby tools have.

Alternatives and trade-offs

Keeping BT.601 for SD inputs would match the source more closely, but the upscaled output is HD-sized, and HD players assume BT.709. HDR needs extra handling (see the short note on tone mapping).

Interview prep

> **Q: What's a subtle bug you prevented?**
>
> A: A colour shift. Video is stored as YUV, and the models need RGB. ffmpeg's default conversion uses the SD colour matrix, but HD players assume BT.709, so colours would drift. I set the matrix explicitly in both directions, tag the output, and my test measured an average colour shift of only about one level out of 255.

Future use

Any image or video pipeline, camera apps, and computer vision (OpenCV loads images as BGR, not RGB, which is the same type of bug). Always know which colour format your data is in.

35. Resampling: make it 4× bigger, then shrink carefully

★★ Important · Category: image processing

What it is

Analogy: drawing a picture extra large and then reducing it on a photocopier. The small copy looks cleaner than drawing it small directly.

Technically: resampling means changing an image's size. The models always produce **4×**. For `SCALE = 2`, the code then **downscales** 4× → 2× with **bicubic interpolation and antialiasing**. That's a form of **supersampling**: computing more detail than needed, then averaging it down. Antialiasing blurs very slightly before shrinking, to avoid jagged edges and shimmering (called *aliasing*).

Where it is in your project

`dev/engine.py` · lines 802–807 · shrink with antialias, then blend with the original

```
802         if sr.shape[-2:] != (self.out_h, self.out_w):
803             sr = F.interpolate(sr, size=(self.out_h, self.out_w), mode="bicubic",
804                                align_corners=False, antialias=True)
805         if self.blend > 0:
806             sr = sr * (1 - self.blend) + self.original_upscaled(lr) * self.blend
807         sr = sr.clamp(0, 1)
```

Why we used it

These models only exist in 4× versions. And shrinking from 4× hides some of the model's artefacts, which gives a more natural look. That's partly why `SCALE = 2` is recommended.

How it works

1. `F.interpolate(sr, size=(out_h, out_w), mode="bicubic", antialias=True)` resizes on the GPU.
2. `original_upscaled(lr)` is the **plain bicubic enlargement** of the input. It's used for `ORIGINAL_BLEND` and for the "BEFORE" side of the preview.
3. Blending is a weighted average: `sr*(1-blend) + original*blend`.
4. `.clamp(0, 1)` keeps all values within valid brightness.

What would go wrong without it

Without antialiasing, downscaled fine patterns (like fabric or brick walls) shimmer and get jagged. Using nearest-neighbour resizing gives blocky edges.

Alternatives and trade-offs

Lanczos resizing is a little sharper (Real-ESRGAN's own code uses it), but can create halos. Area averaging is softer. Running the model at 4× and shrinking takes about 4× more compute for the final output stage than a native 2× model would, but no native 2× version of these models exists.

Interview prep

> **Q: Your model is 4×. How do you produce 2×?**
> A: I run the 4× model, then downscale on the GPU with bicubic interpolation and antialiasing. That supersampling reduces artefacts and looks more natural than upscaling directly to 2×.

Future use

Use it for thumbnails, responsive images (serving smaller versions to phones), game rendering (supersampling for anti-aliasing), and generating icons.

36. Generated notebook: a single source of truth

★★ Important · Category: tooling / build process

What it is

Analogy: a recipe book printed from one master document. If you fix a typo, you fix the master and reprint. You don't correct every printed copy by hand.

Technically: the notebook (`video_upscaler.ipynb`) is **generated** by a script from `engine.py` plus the cell texts. `engine.py` is the **single source of truth**: the one place the code really lives. A notebook file is JSON, which is hard to read, diff and test, while a `.py` file is easy.

Where it is in your project

dev/build_notebook.py · lines 134–142 · helper functions that create notebook cells

```
134 def md(text):
135     return {"cell_type": "markdown", "metadata": {}, "source": text.splitlines(keepends=True)}
136
137
138 def code(text, hide=False):
139     meta = {"_kg_hide-input": True, "jupyter": {"source_hidden": True}} if hide else {}
140     return {"cell_type": "code", "execution_count": None, "metadata": meta, "outputs": [],
141             "source": text.splitlines(keepends=True)}
```

dev/build_notebook.py · lines 144–163 · build the notebook: engine.py becomes the ENGINE cell

```
144 def main():
145     engine = open(os.path.join(HERE, "engine.py"), encoding="utf-8").read()
146     nb = {
147         "nbformat": 4, "nbformat_minor": 5,
148         "metadata": {
149             "kernelspec": {"display_name": "Python 3", "language": "python", "name": "python3"},
150             "language_info": {"name": "python"},
151             "kaggle": {"accelerator": "nvidiaTeslaT4", "isInternetEnabled": True, "isGpuEnabled":
True},
152         },
153         "cells": [md(INTRO), md(SETTINGS_MD), code(SETTINGS), md(ENGINE_MD), code(engine),
154                   md(CHECK_MD), code(CHECK), md(PREVIEW_MD), code(PREVIEW), md(UPSCALE_MD),
code(UPSCALE),
155                   md(CLEAN_MD), code(CLEAN)],
156     }
157     for i, c in enumerate(nb["cells"]):
158         c["id"] = f"cell-{i:02d}"
159     out = os.path.join(ROOT, "video_upscaler.ipynb")
160     with open(out, "w", encoding="utf-8") as f:
161         json.dump(nb, f, indent=1, ensure_ascii=False)
162         f.write("\n")
163     print("wrote", out)
```

Why we used it

`test_local.py` can `import engine` and test the *exact* code that goes into the notebook. Git diffs show readable Python changes instead of JSON noise.

How it works

1. Read `engine.py` as text.
2. Build a Python dictionary in the notebook format (nbformat 4): `cells`, each with `cell_type` and a `source` list of lines.

3. Add Kaggle metadata (GPU accelerator, Internet). These are hints. You still switch them on in Kaggle's settings.
4. `json.dump(...)` to `video_upscaler.ipynb`.

What would go wrong without it

The notebook and the tested code would drift apart: you'd fix a bug in one and forget the other, and the tests would prove nothing about what users actually run.

Alternatives and trade-offs

Jupytext (keeps notebooks as paired `.py` files), or a notebook cell that `pip install`s your code from GitHub. *Weaknesses:* nothing *checks* that the committed notebook is up to date. A test or CI step could rebuild it and compare. Also, `code(..., hide=True)` is an option that's never used, so it's dead code.

Interview prep

> **Q: Why is there a build script for a notebook?**
> A: So the real code lives in one normal Python file that I can test and review. The notebook is generated from it, so what users run is exactly what the tests ran. If I had more time, I'd add a CI check that the notebook matches the source.

Future use

Generating **API clients from an OpenAPI spec**, documentation from code comments, database models from a schema, config files from templates, or any file that duplicates something else. Generate it, don't hand-maintain it.

37. The Kaggle platform model (and environment detection)

★★ Important · Category: deployment / platform

What it is

Analogy: renting a fully equipped kitchen by the hour. You have to know the house rules: where the ingredients are, where to put the food, and when they lock the doors.

Technically: Kaggle runs your notebook in a container (a pre-packaged computer environment) with rules: **inputs** are read-only in `/kaggle/input`, **outputs** go to `/kaggle/working` (about 20 GB, the "Output" panel), there's a **12-hour** limit, a **weekly GPU quota**, and two ways to run: **interactive** or background **"Save Version"** runs (batch). Your code **detects** where it runs and adapts.

Where it is in your project

dev/engine.py · lines 21–25 · detect Kaggle and its folders and limits

```
21 ON_KAGGLE = os.path.isdir("/kaggle/working")
22 MODEL_DIR = "/tmp/upscaler_models"            # model files (not part of your output)
23 WORK_FOLDER_NAME = "upscaler_work"            # saved progress lives here
24 PREVIEW_FOLDER_NAME = "previews"
25 KAGGLE_OUTPUT_LIMIT_GB = 19.5                 # Kaggle keeps ~20 GB in /kaggle/working
```

dev/engine.py · lines 1351–1353 · skip the preview in background runs

```
1351     if S.SKIP_PREVIEW_IN_SAVE_VERSION and os.environ.get("KAGGLE_KERNEL_RUN_TYPE", "").lower() ==
"batch":
1352         print("⏭  Preview skipped (this is a 'Save Version' background run). Going on to the full
upscale.")
1353         return
```

dev/build_notebook.py · lines 151–151 · notebook metadata asking for a T4 and Internet

```
151             "kaggle": {"accelerator": "nvidiaTeslaT4", "isInternetEnabled": True, "isGpuEnabled":
True},
```

Why we used it

"Deployment" for this project means "runs correctly on Kaggle". Knowing the platform rules shaped every major design decision: streaming (disk limit), checkpointing (time limit), carrying progress over between versions (outputs become inputs), and skipping previews in batch mode.

How it works

- `ON_KAGGLE = os.path.isdir("/kaggle/working")` is simple feature detection.
- `KAGGLE_KERNEL_RUN_TYPE` is an environment variable Kaggle sets. The code treats `"Batch"` as a background run. *Untested assumption*: it was never verified on Kaggle, and if it's wrong, the preview just runs for a few minutes.
- Model files go to `/tmp` so they don't use up your output quota.
- `find_videos` searches all of `/kaggle/input`, but skips your own earlier outputs.

What would go wrong without it

The disk would fill up, files would be lost when the session ends, previews would waste batch time, and there'd be no resume across versions.

Alternatives and trade-offs

Google Colab (similar free GPU, different rules), or a paid cloud GPU (RunPod, Modal) with no time limits but a cost. Kaggle was chosen because it's free.

Interview prep

> **Q: How did the platform shape your design?**
> A: Kaggle gives you a free T4 but a 20 GB output limit, a 12-hour session limit and a weekly quota. So I stream frames instead of saving them, checkpoint every 10 seconds of video, stop safely before the time limit, and let a new run pick up the previous run's output. Platform limits drove the architecture.

Future use

Every hosting platform has rules: Heroku (filesystem wiped on restart), AWS Lambda (15-minute limit, read-only disk except `/tmp`), Vercel (function timeouts), and mobile OSes (background limits). Read the limits first, then design.

38. Installing packages at runtime, version pinning and patching third-party code

★★ Important · Category: dependencies

What it is

Analogy: your recipe needs a specific brand of flour (version 1.4.2), and the packet has a misprint in its instructions. You buy that exact packet and correct the misprint with a pen before using it.

Technically: dependencies are other people's code your project uses. **Pinning** means requiring an exact version (`gfpgan==1.3.8`) so updates can't break you. Here the code also **patches** (edits) a broken old library, `basicsr 1.4.2`, before installing it, because it fails with new Python and torchvision.

Where it is in your project

`dev/engine.py` · lines 1081–1106 · download basicsr's source, fix two known bugs, install pinned versions

```
1081     try:
1082         # basicsr 1.4.2 (needed by GFPGAN) has two known bugs with newer Python/torchvision.
1083         # We download its source, fix those two lines, and install the fixed copy.
1084         meta = json.load(urllib.request.urlopen("https://pypi.org/pypi/basicsr/1.4.2/json",
timeout=60))
1085         url = [u["url"] for u in meta["urls"] if u["filename"].endswith(".tar.gz")][0]
1086         tmp = tempfile.mkdtemp()
1087         tgz = os.path.join(tmp, "basicsr.tar.gz")
1088         urllib.request.urlretrieve(url, tgz)
1089         with tarfile.open(tgz) as t:
1090             t.extractall(tmp)
1091         src_dir = os.path.join(tmp, "basicsr-1.4.2")
1092         setup_py = os.path.join(src_dir, "setup.py")
1093         s = open(setup_py).read()
1094         s = s.replace("return locals()['__version__']", "return '1.4.2'")
1095         s = s.replace("setup_requires=['cython', 'numpy', 'torch'],", "")
1096         open(setup_py, "w").write(s)
1097         deg = os.path.join(src_dir, "basicsr", "data", "degradations.py")
1098         d = open(deg).read().replace("torchvision.transforms.functional_tensor",
1099                                      "torchvision.transforms.functional")
1100         open(deg, "w").write(d)
1101     except Exception as e:
1102         raise UpscalerError(f"Could not download GFPGAN's helper package. Is Internet turned on?
\n{e}\n"
1103                             "You can set FACE_RESTORE = False and continue.")
1104     steps = [pip + ["addict", "yapf", "lmdb", "filterpy", "scikit-image"],
1105              pip + ["--no-deps", "--no-build-isolation", src_dir],
1106              pip + ["--no-deps", "facexlib==0.3.0", "gfpgan==1.3.8"]]
```

Why we used it

GFPGAN needs `basicsr`, which is no longer maintained. During development its install script failed on Python 3.13 (`locals()['__version__']` behaves differently now), and it imports a torchvision module that was renamed. Patching was the only way to keep the optional face feature working.

How it works

1. Ask PyPI's JSON API for the source download link of `basicsr 1.4.2`.
2. Download and extract it to a temporary folder.
3. Replace the broken version-reading line with `return '1.4.2'`, and remove the `setup_requires` line.

4. Replace `torchvision.transforms.functional_tensor` with `torchvision.transforms.functional`.
5. `pip install --no-deps --no-build-isolation <folder>`. `--no-deps` stops pip from upgrading or downgrading PyTorch, which could break Kaggle's setup.
6. Install the pinned `facexlib==0.3.0 gfpgan==1.3.8`, also with `--no-deps`.
7. `importlib.invalidate_caches()` so Python notices the new packages without a restart.

What would go wrong without it

The face feature wouldn't install at all, or a normal `pip install gfpgan` could replace Kaggle's PyTorch with an incompatible version and break *everything*.

Alternatives and trade-offs

Fork the library on GitHub and install from your fork (cleaner), find a maintained alternative, or drop the feature. *Weaknesses* (see the security review): the helper packages `addict, yapf, lmdb, filterpy, scikit-image` are **not pinned**, the download isn't checksum-verified, and patching by text replacement is fragile.

Interview prep

> **Q: Tell me about a dependency problem you solved.**
> A: The face-restoration library depends on basicsr, which is unmaintained and broken on new Python and torchvision. I download its source, patch the two broken lines, and install it with no-deps so it can't change PyTorch. I pin the other packages to exact versions. It's optional and only installed when enabled.

Future use

Use **lock files** (`requirements.txt` with exact versions, `package-lock.json`, `poetry.lock`) in every project. When a library is abandoned: patch carefully, fork it, or replace it, and write down why.

39. Resource checks: disk space and quotas

★★ Important · Category: reliability

What it is

Analogy: checking you have enough petrol before a long drive, not when the engine starts coughing.

Technically: before expensive steps, the code estimates how much disk space it will need and compares that with what's free, including Kaggle's own **quota** for `/kaggle/working`, which can be smaller than what the disk reports.

Where it is in your project

dev/engine.py · lines 1218–1223 · free space = the smaller of disk free and the Kaggle quota

```
1218 def free_space(path):
1219     os.makedirs(path, exist_ok=True)
1220     free = shutil.disk_usage(path).free
1221     if ON_KAGGLE and os.path.abspath(path).startswith("/kaggle/working"):
1222         free = min(free, int(KAGGLE_OUTPUT_LIMIT_GB * 1024 ** 3) - folder_size("/kaggle/working"))
1223     return max(0, free)
```

dev/engine.py · lines 1598–1602 · before each part: enough room for ~3 parts + 300 MB?

```
1598                 need = (estimate_mb_per_second(out_w, out_h, S) * float(S.SAVE_EVERY_SECONDS) * 3
1599                         * 1024 ** 2 + 300 * 1024 ** 2)
1600                 if free_space(out_dir) < need:
1601                     status = "disk"
1602                     break
```

dev/engine.py · lines 1647–1651 · before joining: room for a full copy of all parts?

```
1647     need = folder_size(work) * 1.05 + 200 * 1024 ** 2
1648     if free_space(out_dir) < need:
1649         raise UpscalerError(f"Not enough disk space to join the parts (needs about
{human_size(need)}).\n"
1650                             "Download and delete other finished videos (or previews), then run this
cell again.\n"
1651                             "Your progress is saved.")
```

Why we used it

A full disk in the middle of encoding gives cryptic ffmpeg errors and corrupt files. Stopping *before* that, with a clear message and saved progress, turns a crash into a pause.

How it works

1. `shutil.disk_usage(path).free` is what the operating system reports.
2. On Kaggle, it also calculates `19.5 GB − size of everything in /kaggle/working` and takes the **smaller** of the two.
3. Segment size is estimated with a rough formula (`estimate_mb_per_second`). The check asks for 3× that plus a buffer.
4. Joining needs about the same space again as all the parts, so it's checked separately.
5. The CHECK cell warns early if a video probably won't fit.

What would go wrong without it

Corrupted outputs, confusing errors, and lost work right at the end of a long run (joining is the biggest single disk-space moment).

Alternatives and trade-offs

The size estimate is **rough**: real file sizes depend on the content (grainy video is much bigger). Being too cautious stops runs early, while being too optimistic risks a full disk. Ffmpeg's own error is still caught as a last line of defence (the `VideoWriter` error message mentions disk space).

Interview prep

> **Q: How do you handle running out of disk?**
> A: I check before each segment and before the final join, using the smaller of the real free space and Kaggle's output quota. If it's too low, I stop with progress saved and tell the user what to delete, instead of crashing halfway through a write.

Future use

Use it for upload size limits, database storage alerts, log rotation (deleting old logs before the disk fills up), mobile apps (checking storage before downloads), and memory budgets in caches.

Part 2C — Basics (★)

Shorter notes. Each one covers what it is, where it is, why it's there, and one interview line.

40. Callable objects (`__call__`)

★ Basic · Python

What: a class with a `__call__` method can be *called like a function*: `finish(sr, lr)`. It's like a tool that remembers its settings. **Where:** `Finisher.__call__` (engine.py 798), `FaceRestorer.__call__` (1052). **Why:** the finisher needs settings (output size, blend, grain, a blur kernel) prepared once, and then a simple "do it" call for every frame. **Without it:** you'd pass five settings into every call, or use global variables. **Interview line:** "Finisher is a callable object. It's configured once per video and then used like a function on each frame."

dev/engine.py · lines 786–792 · set up once…

```
786     def __init__(self, out_w, out_h, S, device):
787         self.out_w, self.out_h = out_w, out_h
788         self.blend = float(S.ORIGINAL_BLEND)
789         self.grain = float(S.FILM_GRAIN)
790         self.device = device
791         k = torch.tensor([0.25, 0.5, 0.25], device=device)
792         self.grain_kernel = (k[:, None] * k[None, :]).view(1, 1, 3, 3)
```

41. Exact fractions for frame rates

★ Basic · Python / video

What: `fractions.Fraction` stores numbers like 30000/1001 *exactly*, while a float stores 29.97002997… with tiny rounding errors. **Where:** `parse_rate` (engine.py 201) and `probe_video`. **Why:** "29.97 fps" is really 30000/1001. Over an hour, a rounded rate drifts by frames, so the audio goes out of sync. ffmpeg accepts the exact `"30000/1001"` text. **Interview line:** "I keep frame rates as exact fractions to avoid audio drift from rounding."

42. Peek / push-back buffer (`deque`)

★ Basic · data structures

What: a `deque` ("deck") is a list that adds and removes quickly at both ends. **Where:** `FrameReader.pending` and `peek_n` (engine.py 381–384). **Why:** the steady model needs to *look ahead* at the next 5 frames without consuming them. They're read, then pushed back to the front with `extendleft(reversed(frames))`. **Without it:** the look-ahead frames would be lost or read twice from ffmpeg. **Interview line:** "I implemented peeking on a stream with a push-back deque."

43. Logging to files and showing the tail

★ Basic · debugging

What: ffmpeg's error output goes to a `.log` file, and `log_tail` reads the last ~600 bytes when something fails. **Where:** engine.py 311–319, used in error messages. **Why:** the user sees *the actual reason* ("No space left on device") without a wall of text. **Weakness:** the project uses `print` instead of Python's `logging` module, so there are no log levels (info/warning/error) and no timestamps. **Interview line:** "External tool errors are captured to a log, and the relevant tail is shown in a friendly message."

dev/engine.py · lines 311–319

```
311 def log_tail(path, n=600):
312     try:
313         with open(path, "rb") as f:
314             f.seek(0, 2)
315             size = f.tell()
316             f.seek(max(0, size - n))
317             return f.read().decode("utf-8", "replace").strip()
318     except Exception:
319         return ""
```

---

44. `cudnn.benchmark`

★ Basic · GPU performance

What: `torch.backends.cudnn.benchmark = True` (engine.py 28) lets NVIDIA's cuDNN library try several ways of computing a convolution the first time, and then keep using the fastest. **Why:** video frames all have the same size, so the one-time test pays off thousands of times. **Trade-off:** if sizes change often (like tiles at the image edges), it re-tests for each new size, which is a small cost.

---

45. Skipping wasted work

★ Basic · performance

What: in `RealBasicVSRNet.run` (engine.py 731 and 740), the backward pass stops at the first frame that's needed, the forward pass stops after the last one, and the expensive 4× upscaling only runs for **core** frames, never for context frames. **Why:** context frames are only there to give information. Upscaling them would be thrown away. **Interview line:** "I avoided computing outputs nobody will use: context frames only feed the propagation."

---

46. Tensor layout and normalisation

★ Basic · machine learning

What: images from ffmpeg are `(height, width, 3)` bytes from 0–255. PyTorch models expect `(batch, channels, height, width)` decimals from 0–1. **Where:** `frames_to_tensor` (engine.py 818–820), and the reverse in `Finisher` (line 815: `*255`, `+0.5` to round, `clamp`, `uint8`, `permute`). **Why:** a wrong layout or range gives garbage colours, or an error. **Interview line:** "I convert HWC uint8 to NCHW float in 0–1 on the way in, and round and clamp back on the way out."

dev/engine.py · lines 818–820

```python
def frames_to_tensor(frames, device):
    arr = torch.from_numpy(np.stack(frames))                # T,H,W,3 uint8
    return arr.to(device).permute(0, 3, 1, 2).float().div_(255.0)
```

47. Frame-exact seeking with ffmpeg's `select` filter

★ Basic · video

What: `select=gte(n\,START)` keeps only frames whose number is ≥ START. `setpts=N/FRAME_RATE/TB` then renumbers their timestamps. **Where:** `decode_command` (engine.py 298–299). **Why:** normal time-based seeking (`-ss`) can be off by a frame, and resume needs *exact* frames, or you'd get a duplicated or missing frame at the join. The test proves it's exact. **Trade-off:** ffmpeg still decodes the skipped frames, which is slower for resuming near the end of long videos. **Interview line:** "Resume uses frame-number selection, not time seeking, so the join is frame-exact."

48. Constant vs variable frame rate (CFR / VFR)

★ Basic · video

What: phones often record with a *variable* frame rate (VFR), meaning frames aren't evenly spaced. **Where:** `fps=` filter in `decode_command` (line 297), and the `vfr` flag in `probe_video` (line 279). **Why:** the encoder and the segment maths assume evenly spaced frames. The `fps` filter makes the stream constant (CFR) by repeating or dropping the odd frame. **Interview line:** "Variable-frame-rate phone videos are normalised to a constant rate, so segment boundaries and audio sync stay correct."

49. HDR tone mapping

★ Basic · video

What: HDR video (for example, iPhone HLG) has a much wider brightness range than normal SDR. Shown as SDR without conversion, it looks grey and washed out. **Tone mapping** squeezes the range into SDR. **Where:** engine.py 301–303 (the `zscale` + `tonemap=hable` chain, used only if the ffmpeg build supports it). **Why:** iPhones record HDR by default. **Trade-off:** the output is SDR, so the HDR "pop" is lost, but the colours look correct everywhere.

50. Rotation metadata

★ Basic · video

What: phones often save upright videos as *sideways* pictures plus a "rotate 90°" note. **Where:** `probe_video` (engine.py 230–237). **Why:** ffmpeg turns the frames upright automatically when decoding, so width and height must be **swapped** too. Otherwise the code would read the wrong number of bytes per frame and the pictures would be garbled. The test covers this.

51. Film grain synthesis

★ Basic · image processing

What: adds random noise that looks like film grain. **Where:** `Finisher.__call__` (engine.py 808–814). **How:** random noise (`randn`) → a soft 3×3 blur (so the grains are slightly clumped, like real film) → normalised → stronger in mid-tones and weaker in shadows and highlights (`weight = 0.35 + 2.6·L·(1–L)` , where L is brightness) → added the same to all colours. New noise every frame, like real film. **Why:** a little grain hides the "too clean, AI" look and the smoothing from compression.

dev/engine.py · lines 808–814

```
808         if self.grain > 0:
809             noise = torch.randn(1, 1, self.out_h, self.out_w, device=sr.device)
810             noise = F.conv2d(F.pad(noise, (1, 1, 1, 1), mode="reflect"), self.grain_kernel)
811             noise = noise / noise.std().clamp_min(1e-6)       # soft, film-like grain clumps
812             luma = (0.2126 * sr[:, 0:1] + 0.7152 * sr[:, 1:2] + 0.0722 * sr[:, 2:3])
813             weight = 0.35 + 2.6 * luma * (1 - luma)           # grain shows most in mid-tones
814             sr = (sr + noise * weight * self.grain).clamp(0, 1)
```

52. Alpha blending and before/after comparison

★ Basic · image processing / evaluation

What: *alpha blending* is a weighted average of two images (`a·(1–t) + b·t`). **Where:** `ORIGINAL_BLEND` in `Finisher` (line 806), `FACE_STRENGTH` in `FaceRestorer` (1068), and the preview's `side_by_side` (1320–1331), which puts the *same* centre crop of BEFORE and AFTER next to each other at full size. **Why:** blending reduces the AI look and flicker. Side-by-side at 100% is the honest way to judge skin texture (an A/B test, comparing two versions directly). **Interview line:** "I built an A/B preview so settings are judged on real footage before spending GPU hours."

53. Progress bars with time remaining (`tqdm`)

★ Basic · user experience

What: `tqdm(total=..., initial=start)` draws a progress bar and estimates the time left from the speed so far. **Where:** engine.py 1392 and 1588. **Why:** long runs need feedback. `initial=start` makes a resumed run show the correct percentage, and `smoothing=0.05` steadies the time estimate. The preview also extrapolates the **full-video time** from its own speed (line 1440).

54. Human-friendly formatting helpers

★ Basic · user experience

What: `human_time` ("2 h 15 min") and `human_size` ("3.20 GB") in engine.py 62–75, plus `say()` , which prints boxed messages. **Why:** the audience is beginners. Small helpers keep every message consistent. **Interview line:** "Small, reusable helpers for user-facing text."

55. Git basics: `.gitignore`, branches, commit messages

★ Basic · version control

What: `.gitignore` lists files Git should never save: `__pycache__/`, `*.pyc` (Python's compiled cache) and `.ipynb_checkpoints/` (Jupyter's autosaves). The work was done on a **feature branch** (`ccr-a1bffdba-y5q3kp`) and copied to `main`. The commit message has a short title plus bullet points explaining *what* and *why*. **Why:** a clean history and no junk files. **Improvements:** smaller commits (one per feature), tags for versions (`v1.0`), and pull requests with reviews.

.gitignore · lines 1–3

```
__pycache__/
*.pyc
.ipynb_checkpoints/
```

Part 3 — Honest review

Security review

Your project has a small **attack surface**, meaning the number of ways in for an attacker. There's no website, no login, no database and no open network port. Its risks come from **running other people's files and code**: model files, downloaded packages, and the videos themselves. Interviewers *love* asking "what are the weak spots?", so know these.

What it does well

- **No shell injection.** Every ffmpeg command is passed as a **list of arguments** (`subprocess.run(["ffmpeg", "-i", path, ...])`), never as one text string to a shell. A video called `my video; rm -rf /.mp4` is just a file name, never a command. The one `shell=True` call (`apt-get install ffmpeg`, engine.py 1259) is a fixed text with no user input in it.
- **No secrets anywhere.** There are no API keys or passwords in the code, and all downloads are public.
- **Input validation** of every setting and file before work starts (concept 7).
- **Safe model loading first:** `torch.load(..., weights_only=True)` refuses to run code hidden in a model file (engine.py 541).
- **HTTPS downloads from official sources only** (GitHub releases, OpenMMLab, PyPI), with a minimum-size check and atomic rename.
- **`--no-deps` installs**, so the optional face add-on can't silently replace PyTorch.
- **Writes stay inside known folders** (`/kaggle/working`, `/tmp`), and the code never reads its own outputs as inputs.

Weak spots (be ready to talk about these)

#	Weakness	Where	Why it matters	Fix
1	**Unsafe model-loading fallback:** if `weights_only=True` fails, it retries with `weights_only=False`.	engine.py 539–544	`.pth` files use Python's *pickle* format, which can contain **code that runs when loaded**. It's worse because `get_weights` automatically uses **any** `.pth` with "basicvsr" in its name found in `/kaggle/input`, so a malicious shared dataset could run code in your session.	Convert the RealBasicVSR checkpoint once into a safe format (**safetensors**) or a plain tensor dict, then always load with `weights_only=True`. Only use files you chose explicitly.
2	**No checksum verification** of downloads.	engine.py 515–527	If a download is corrupted or tampered with, the code can't tell. The size check is not a security check.	Store the expected **SHA-256** of each file and compare after downloading.

#	Weakness	Where	Why it matters	Fix
3	**`tarfile.extractall` without a filter.**	engine.py 1089–1090	A crafted archive could write files *outside* the target folder ("tar slip", a path-traversal attack). Python 3.12+ warns about this.	Use `t.extractall(tmp, filter="data")`.
4	**Unpinned packages:** `addict, yapf, lmdb, filterpy, scikit-image` install the *latest* version.	engine.py 1104	A future broken or malicious release would be installed automatically (a *supply-chain* risk).	Pin exact versions, ideally with hashes (`pip install --require-hashes`).
5	**Patching third-party code by text replacement.**	engine.py 1093–1100	If the file content differs, the patch silently does nothing, and the error appears later.	Check that each replacement actually happened, or fork the library.
6	**Browser-like User-Agent** (`Mozilla/5.0`).	engine.py 520	Not dangerous, but it pretends to be a web browser. Honest automation should identify itself.	Use something like `ContentUP-upscaler/1.0`.
7	**Privacy of your videos.**	Kaggle settings	If the dataset or notebook output is made **public**, your personal videos are public too.	Keep datasets **Private** (the README says so).

Bugs and code-quality weak spots (not security)

- **Glob special characters:** file names containing `[` or `]` break the `glob.glob(...)` patterns in `bring_back_earlier_work` and `process_video` (lines 1482, 1490, 1549). Resume-from-earlier-run wouldn't find the files. *Fix:* `glob.escape(name)`.
- **Dead code:** `results = {}` in `run_upscale` is filled but never used (line 1669). The `hide` option in `build_notebook.code()` is never used.
- **`@friendly` returns `None` on error**, so a caller can't tell failure from success (fine today, because nobody checks).
- **The resume decode cost:** restarting at frame N decodes all frames before N again (slow near the end of very long videos).
- **Only one of the two T4 GPUs is used.**
- **Heuristics that weren't tested on Kaggle itself:** the session start time (psutil PID 1) and `KAGGLE_KERNEL_RUN_TYPE == "Batch"`.
- **Not tested on a real GPU, or with the real RealBasicVSR weights.** The steady model's code matches the official source and loads strictly, but its output quality and speed are unverified.
- **No dependency file, no CI, no pytest.** The single test script stops at the first failure.
- **One 1,700-line file**, module-level global state (`_ENGINE_CACHE`), `print` instead of `logging`, and no type hints.

Things I could improve ("If I had more time, I would...")

1. **...use both T4 GPUs.** Run one engine per GPU on alternating batches, for almost twice the speed on Kaggle's "T4 x2".

2. **…harden model loading.** Convert every checkpoint to safetensors, verify SHA-256 checksums, and remove the `weights_only=False` fallback.
3. **…add a `requirements.txt` / lock file** with exact versions, and pin the optional add-on's packages too.
4. **…move the tests to pytest and run them in GitHub Actions** on every push, including a check that `video_upscaler.ipynb` matches `engine.py`.
5. **…split `engine.py` into modules** (`settings.py`, `video_io.py`, `models.py`, `engines.py`, `pipeline.py`). The build script would combine them into the notebook. I'd add type hints, a `@dataclass` for settings, and an abstract `Engine` base class.
6. **…use Python's `logging` module** with levels and timestamps, and write a run log into the output.
7. **…make resume faster** by seeking to the nearest keyframe first and then selecting exact frames.
8. **…reduce flicker in "fast" mode** cheaply, for example by blending with a motion-compensated previous frame.
9. **…add more model choices**, like a stronger "detail" model (Real-ESRGAN x4plus) for very low-resolution sources, behind the same Strategy interface.
10. **…measure quality, not just correctness.** A benchmark notebook reporting frames/second, a perceptual quality score (e.g. LPIPS) and a flicker score, so settings decisions are data-driven.
11. **…offer NVENC hardware encoding** as a speed option for drafts.
12. **…fix the small bugs** listed above (`glob.escape`, `filter="data"`, dead code).

Part 4 — Interview kit

Explain your project in 60 seconds

> "ContentUP is a free video upscaler I built to run on Kaggle's free T4 GPU. The goal is upscaling that looks **natural**, not over-sharpened or plastic.
>
> You upload a video and run a few notebook cells. A settings cell is validated up front. Then ffmpeg decodes the video into raw frames through a **pipe**, PyTorch upscales them on the GPU, and a second ffmpeg encodes them to H.264. All three run **in parallel**, using threads and bounded queues, and no frames are written to disk.
>
> There are two models behind one interface: Real-ESRGAN for speed, with an adjustable denoise made by **blending two models' weights**, and RealBasicVSR, which uses **optical flow** across neighbouring frames to reduce flicker. If the GPU runs out of memory, it automatically falls back to smaller batches and then seamless **tiles**.
>
> Kaggle limits sessions to 12 hours, so it **checkpoints** every 10 seconds of video with **atomic writes**, stops safely before the limit, and resumes frame-exactly. That's verified by a test proving that a resumed video is identical to an uninterrupted one. There's also a **preview mode** that renders a before/after clip and estimates the full run time.
>
> I built it with an AI assistant, then studied and tested every part. If I had more time, I'd use both GPUs, harden model loading with checksums and safetensors, and add CI."

Tips: practise it out loud until it takes about 60 seconds. Stop after any sentence if the interviewer jumps in. Each **bold** word is a doorway to a concept in this guide, so be ready to go deeper on whichever one they pick.

Likely follow-up questions (with the concept number to revise):

Question	Concept
"Walk me through what happens when you run it."	Journey of a request, 1, 2
"What was the hardest problem?"	3 (resume) or 5 (memory)
"How do you know it works?"	12 (tests)
"What would you do differently?"	Part 3 improvements
"What are the security risks?"	Security review
"Why not a web app?"	"What is NOT in this project"
"How did you deal with dependencies breaking?"	11, 38

Glossary

Alpha blending: mixing two images with a weight, e.g. 90% of one and 10% of the other.
Antialiasing: slightly softening an image before shrinking it, so fine patterns don't turn jagged or shimmer.
API: a defined way for one piece of software to ask another for something (e.g. PyPI's JSON API).
Argument list (subprocess): giving a command as separate pieces (`["ffmpeg", "-i", file]`) so nothing in a file name can be run as a command.
Assertion (`assert`): a line in a test that stops with an error if something isn't true.

Atomic operation: something that happens completely or not at all, with no half-done state.
Attack surface: all the ways an attacker could get into or misuse a system.
autocast: a PyTorch feature that automatically runs safe operations in half precision.
Backpressure: slowing down a fast producer so it can't overwhelm a slow consumer.
Base case: the condition where a recursive function stops calling itself.
Batch: several items processed together in one go (e.g. 4 frames in one GPU call).
Batch run / "Save Version": a Kaggle run that executes the whole notebook in the background.
bfloat16 / fp16 / fp32: number formats with 16 or 32 bits. Fewer bits are faster and smaller, but less precise.
BT.601 / BT.709: the standard "dictionaries" for converting colours between RGB and YUV, for SD and HD video.
Cache / caching: keeping a result so it can be reused instead of recomputed.
Callback: a function you hand to other code, for it to call when something happens.
CFR / VFR: constant frame rate (evenly spaced frames) and variable frame rate (uneven, common on phones).
Checkpoint: saved progress that lets a job continue after a stop.
Checksum (SHA-256): a fingerprint of a file, used to check it hasn't been changed or corrupted.
CI (continuous integration): a service that automatically runs your tests on every code change.
Class / object: a class is a blueprint, and an object is one thing built from it.
Closure: a function that remembers variables from the place where it was created.
Codec: the method used to compress video or audio (e.g. H.264, AAC).
Concat demuxer: an ffmpeg feature that reads a list of video files as if they were one.
Container (computing): a packaged, isolated computer environment, like the one Kaggle runs your notebook in.
Container (video): the file format that holds video and audio streams together (e.g. MP4).
Context frames: extra neighbouring frames given to the model for information, not saved to the output.
Context manager (`with`): Python syntax that sets something up and always cleans it up afterwards.
Convolution: the basic operation of image AI: sliding a small grid of learned weights over the picture.
CPU / GPU: the general-purpose processor, and the graphics processor that does thousands of small calculations at once.
CRF: x264's quality setting. Lower is better quality and a bigger file.
CUDA / cuDNN: NVIDIA's software for running calculations on its GPUs, and its fast neural-network library.
Daemon thread: a background thread that doesn't stop Python from exiting.
Dataset (Kaggle): a folder of files you upload to Kaggle, which appears under `/kaggle/input`.
Deadlock: two parts of a program each waiting for the other forever.
Decoder / encoder: a decoder turns compressed video into pictures, and an encoder does the reverse.
Decorator: a function that wraps another function to add behaviour (`@friendly`).
Defensive parsing: reading external data while assuming any field may be missing or wrong.
Dependency: someone else's code that your project needs.
deque: a list that is fast to add to and remove from at both ends.
Device: where a tensor lives and is computed (`cpu` or `cuda:0`).
Divide and conquer: splitting a problem into smaller ones until each is easy.
DNI (network interpolation): blending the weights of two compatible models to get something in between.
Duck typing: treating objects as compatible because they have the same methods, without a formal interface.
EMA weights: a smoothed average of a model's weights from training, which usually gives better results.
End-to-end test: a test that runs the whole program flow from input to output.
Environment variable: a named setting provided by the operating system to a program.
EOF: "end of file", meaning there's no more data to read.
Exception: Python's way of signalling an error, which jumps to the nearest handler.

Exponential backoff: waiting longer and longer between retries (1, 2, 4, 8 seconds).
Facade: a simple front-door function that hides a complicated system.
Factory: a function whose job is to create the right kind of object.
Feature (neural network): the network's internal "notes" about an image, stored as many channels of numbers.
ffmpeg / ffprobe: the standard free tools for converting video and for reading its information.
Frame / fps: one still picture of a video, and how many play per second.
Garbage collector: the part of Python that frees memory nobody uses any more.
GIL: Python's Global Interpreter Lock, which lets only one thread run Python code at a time (it's released while waiting for I/O).
Glob: a file-name pattern with wildcards, like `seg_*.mp4`.
Graceful degradation: automatically falling back to a slower or simpler way instead of failing.
H.264 / libx264: the most compatible video format, and the encoder that produces it.
Hash: a short fixed-length fingerprint calculated from any data.
HDR / SDR / tone mapping: high and standard dynamic range video. Tone mapping converts HDR into SDR.
Heuristic: an educated guess or rule of thumb that usually works.
Idempotent: safe to repeat, because running it twice gives the same result as once.
Inference: using a trained model to make predictions (as opposed to training it).
Interpolation (resizing): calculating new pixel values when changing an image's size (nearest, bilinear, bicubic).
Jitter: a small random extra delay that stops many clients retrying at exactly the same time.
JSON: a simple text format for structured data, used for `progress.json`.
Kernel (Jupyter): the running Python process behind a notebook.
Lazy loading: doing expensive work only when it's actually needed.
Lock file / pinning: recording exact dependency versions so installs are repeatable.
Memoization: caching the result of a function call for the same inputs.
Metadata: data about data, e.g. a video's size, frame rate and rotation.
Mixed precision: using 16-bit numbers where it's safe and 32-bit where it's needed.
Module (Python): one `.py` file of code that can be imported.
Muxing: combining separate video and audio streams into one file.
NaN: "not a number", a broken value produced by an invalid calculation.
nn.Module: the PyTorch base class for every neural network and layer.
Notebook / cell: a document of runnable code boxes, and one of those boxes.
NumPy array / tensor: grids of numbers. A tensor is PyTorch's version, which can live on the GPU.
Optical flow: a map of how far each pixel moved between two frames.
Pickle: Python's format for saving objects. Loading it can run code, so it's unsafe for untrusted files.
Pipe (stdin / stdout / stderr): a data channel between programs: standard input, output and error output.
Pipeline: a chain of steps where each one's output is the next one's input.
PixelShuffle: rearranging many low-resolution channels into a bigger image.
Preset / tune (x264): preset is the speed-versus-file-size trade-off. Tune gives content hints such as "film" or "grain".
Process / subprocess: a running program, and a program started by another program.
Producer–consumer: one part creates work items and another processes them, linked by a queue.
Propagation (video model): passing information from frame to frame, forwards or backwards in time.
PyPI / pip: Python's public package library, and the tool that installs from it.
PyTorch: the deep-learning library used to run the models.
Queue: a thread-safe waiting line: first in, first out.
Receptive field: how far away in the input a pixel can influence the network's output.
Recursion: a function solving a problem by calling itself on a smaller part.

Residual connection: adding a block's input to its output, so the block only learns the change.
Resampling / supersampling: resizing an image, and computing at a higher size before shrinking for quality.
Retry: trying a failed operation again.
RGB / YUV / yuv420p: colour as red-green-blue, or as brightness plus colour-difference. yuv420p is the YUV layout every player supports.
Safetensors: a safe model-file format that can't contain runnable code.
Sentinel: a special value that means "the end" (here `None` in a queue).
Separation of concerns: each part of the code has one clear job.
SHA-1 / SHA-256: hash functions. SHA-256 is the current safe choice for security.
Single source of truth: keeping each piece of information or code in exactly one master place.
state_dict: the dictionary of all learned numbers in a PyTorch model.
Strategy pattern: interchangeable classes with the same interface, chosen at runtime.
Supply-chain attack: an attack that hides malicious code inside a dependency you install.
Tar slip / path traversal: a malicious archive writing files outside the folder you extract it into.
Test double: a fake stand-in for something expensive or unavailable, used in tests.
Thread: a separate line of work running inside the same program.
Tile / halo: a piece of a large image processed on its own, and the extra border read around it.
Traceback: Python's detailed report of where an error happened.
Validation: checking input is correct before using it.
Version control (Git) / branch / commit: saving the history of code changes, a parallel line of work, and one saved change.
Warping: moving an image's pixels according to a motion map.
Weights: the learned numbers inside a neural network.
Window (sliding): processing a long sequence in overlapping pieces.

Cheat sheet — last-minute revision

The project in one line

Free Kaggle-notebook video upscaler. ffmpeg → PyTorch (T4 GPU) → ffmpeg, streamed and checkpointed, tuned for a **natural** look.

File map

- `video_upscaler.ipynb`: the product (generated, don't edit)
- `dev/engine.py`: all the code (single source of truth)
- `dev/build_notebook.py`: engine.py → notebook
- `dev/test_local.py`: CPU end-to-end tests with tiny fake videos

★★★ The twelve to know cold

- **Streaming via pipes**: raw RGB through stdout/stdin, no frame files (20 GB disk limit). `FrameReader`, `VideoWriter`
- **Producer–consumer**: threads + `Queue(maxsize)` = parallel work + backpressure. `None` = sentinel.
- **Checkpointing**: 10 s segments + `progress.json`. Delete half-written parts. Frame-exact restart.
- **Atomic writes**: write `.tmp` / `.part` → `os.replace`.
- **Fallbacks**: out of memory → smaller batch → tiles 768…128. NaN → fp32. Audio copy → AAC → none.
- **Retry + backoff**: 4 tries, 1/2/4/8 s, timeout 60 s. (Weak: retries 404s, no jitter.)
- **Validation**: normalise, convert, range table, report all problems at once.
- **Custom exceptions**: `UpscalerError` = user can fix it. Others = bug, full traceback.
- **Decorator** `@friendly`: one error policy for 4 top-level functions.
- **Strategy**: Fast/Steady engines share `run(before, core, after)`, `context`, `step`. Factory `build_engine`.
- **PyTorch weights**: own copy of the architecture + strict `load_state_dict` (avoids broken basicsr).
- **Testing**: synthetic videos, fake checkpoint (test double), pause/resume = identical output.

★★ One-liners

- **finally / BaseException + re-raise**: always kill ffmpeg and delete the half part.
- **Idempotent**: finished files are skipped, unfinished ones resume.
- **Deadline**: stop if now + 1.3 × average segment time > 11.5 h − 10 min.
- **Settings hash**: sha1(sorted JSON)[:8] + version → work folder.
- **Memoized engine**: reuse the model, free the old one before loading a new one.
- **Lazy**: GFPGAN installed only if `FACE_RESTORE`.
- **Sliding window**: 5 before + core + 5 after (peeked).
- **Tiling + halo**: pad 40/48 px ≥ receptive field, keep the centre.
- **fp16 autocast** + `isfinite` check. Flow maths in fp32.
- **Closure callback** `emit`: finish each frame immediately.
- **Recursion**: splits chunks, and always shrinks towards a floor.
- **DNI**: `d·A + (1−d)·B` weights = denoise slider.
- **Residual** `x + f(x)`, **PixelShuffle** channels → pixels.
- **Optical flow** (SPyNet) + warp + two-way propagation = less flicker.
- **Colour**: decode with the source matrix, encode BT.709 TV range + tags.
- **4× then shrink**: bicubic + antialias (supersampling).
- **H.264**: CRF 17, preset medium, tune film/grain, yuv420p, even sizes, faststart.
- **Concat + mux**: `-f concat`, `-c copy`, `-map 1:a?`.

Key numbers

T4 = 16 GB · Kaggle output ≈ 20 GB · session 12 h (stop at 11.5 h) · ~30 GPU h/week · segment 10 s · queues 24 / 8 · batch 4 · chunk 30 · context 5 · CRF 17 · default scale 2, denoise 0.5, blend 0.10.

Weak spots (say them before they ask)

- `weights_only=False` fallback + auto-picking `.pth` files = pickle code-execution risk
- No SHA-256 checks · `extractall` without a filter · unpinned helper packages
- No requirements file / CI / pytest · one 1,700-line file · global caches
- Only 1 of 2 GPUs · resume re-decodes · `glob` breaks on `[ ]` names
- Never run on a real GPU or with the real RealBasicVSR weights

"If I had more time…"

Two GPUs · safetensors + checksums · lock file · pytest + GitHub Actions · split into modules + type hints · logging · faster resume · quality/flicker benchmarks.

Not in the project (and why that's fine)

No UI, server, auth, database or .env. It's a personal single-process tool. The "database" is `progress.json` with atomic writes.

Pitch keywords

natural look · pipes · threads + bounded queues · two models, one interface · weight blending · optical flow · OOM fallback + tiles · checkpoint + atomic writes · frame-exact resume, tested · preview + ETA · built with AI, verified by tests.